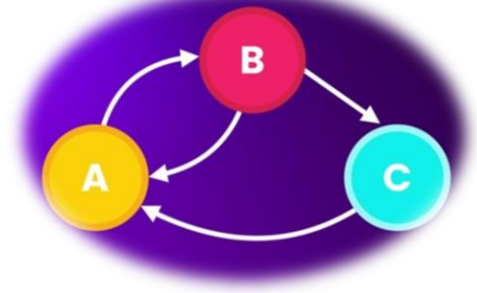


As Per
JNTUA R19
Syllabus

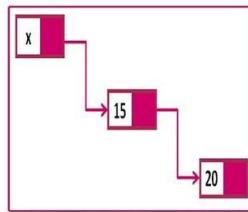
Specially Prepared for Engineering Students



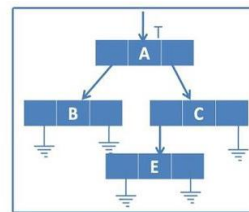
Data Structures



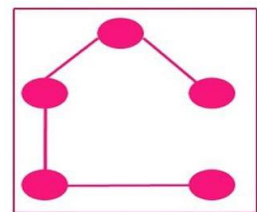
Sorting



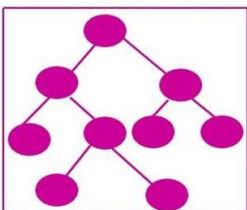
Link list



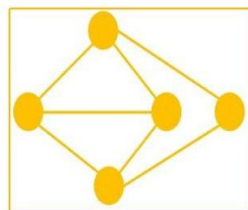
list



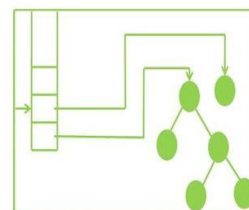
spanning tree



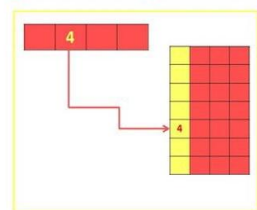
Tree



Graph



Stack



Hashing

By
H. Ateeq Ahmed, M.Tech.,
Assistant Professor of CSE,
Kurnool.

Common to all Branches of JNTUA B.Tech R19 Syllabus

NOT FOR
SALE

ABOUT AUTHOR



H. ATEEQ AHMED completed B.Tech from Safa College of Engineering & Technology, JNTUA, Anantapur, M.Tech from Samskruti College of Engineering & Technology, JNTUH, Hyderabad. Currently he is working as an Asst. Professor in the Department of CSE at Dr. KVSIR Institute of Technology, Kurnool, A.P. He has more than 11 years of teaching experience. He has vast subject knowledge in the field of Computer networks & organization, Network security, Software design & testing, Operating Systems, Distributed systems, Programming in C & Java and Data mining.

His **dynamic** and **illustrative** approach towards the subjects helps the students to enrich their skills in their academics which can help them to achieve success in job career.

He always believes in **real time approach** to solve various problems and expect the same with his students which can help them to understand their subject in an easy and efficient manner.

He has prepared many **Online Video Lectures** through his YouTube channel "**Engineering Drive**" and also **materials** of different subjects for engineering students which can help them to prepare for exams in an **easy** method.

Finally, we expect our students to acquire as much knowledge as possible and convert his **dynamic teaching** into their **individual success**.

ACKNOWLEDGEMENT

First I thank Almighty God for giving me the knowledge to learn and teach various students.

It's my privilege to thanks my parents as without their right guidance and support, this book will be a dream for me.

Finally, I thank my colleagues and friends for helping me during tough period of time.

"A Successful student Watches, Listens and Act"

H. Ateeq Ahmed, M.Tech.,

Mobile no: 994 8 37 8 994,

E-mail ID: ateeqh25@gmail.com.

Website: engineeringdrive.blogspot.in

YouTube Channel:



SYLLABUS & CONTENTS

Topics	Page No.
UNIT-I	
Algorithm Specification, Performance analysis, Performance Measurement. Arrays: Arrays, Dynamically Allocated Arrays. Structures and Unions. Sorting: Motivation, Quick sort, How fast can we sort, Merge sort, Heap sort	1 - 10
UNIT-II	
Stacks, Stacks using Dynamic Arrays, Queues, Circular Queues Using Dynamic Arrays, Evaluation of Expressions, Multiple Stacks and Queues. Linked lists: Singly Linked Lists and Chains, Representing Chains in C, Linked Stacks and Queues, Additional List Operations, Doubly Linked Lists.	11 - 22
UNIT-III	
Introduction, Binary Trees, Binary Tree Traversals, Additional Binary Tree Operations, Binary Search Trees, Counting Binary Trees, Optimal Binary search Trees, AVL Trees. B-Trees: BTrees, B + Trees.	23 – 36
UNIT-IV	
The Graph Abstract Data Type, Elementary Graph Operations, Minimum Cost Spanning Trees, Shortest Paths and Transitive Closure Hashing: Introduction to Hash Table, Static Hashing, Dynamic Hashing.	37 – 52

UNIT-V	
File Organization: Sequential File Organization, Direct File Organization, Indexed Sequential File Organization. Advanced sorting: Sorting on Several keys, List and Table sorts, Summary of Internal sorting, External sorting.	53 – 56

UNIT-I

INTRODUCTION

Algorithm Specification

Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output. Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.

Characteristics of an Algorithm

An algorithm should have the following characteristics –

- **Finiteness** – Algorithms must terminate after a finite number of steps.
- **Definiteness** – Each step of the algorithm must be precisely stated.
- **Effectiveness** – Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
- **Generality**- An algorithm should have step-by-step directions, which should be independent of any programming code.
- **Input/Output** – An algorithm should have 0 or more well-defined inputs and should have 1 or more well-defined outputs.

Performance analysis

If we want to go from city "A" to city "B", there can be many ways of doing this. We can go by flight, by bus, by train and also by bicycle. Depending on the availability and convenience, we choose the one which suits us. Similarly, in computer science, there are multiple algorithms to solve a problem. When we have more than one algorithm to solve a problem, we need to select the best one. Performance analysis helps us to select the best algorithm from multiple algorithms to solve a problem. When there are multiple alternative algorithms to solve a problem, we analyze them and pick the one which is best suitable for our requirements.

The formal definition is as follows...

Performance of an algorithm is a process of making evaluative judgement about algorithms.

It can also be defined as follows...

Performance of an algorithm means predicting the resources which are required to an algorithm to perform its task.

That means when we have multiple algorithms to solve a problem, we need to select a suitable algorithm to solve that problem. We compare algorithms with each other which are solving the same problem, to select the best algorithm. To compare algorithms, we use a set of parameters or set of elements like memory required by that algorithm, the execution speed of that algorithm, easy to understand, easy to implement, etc.,

Generally, the performance of an algorithm depends on the following elements.

- Whether that algorithm is providing the exact solution for the problem?
- Whether it is easy to understand?
- Whether it is easy to implement?
- How much space (memory) it requires to solve the problem?
- How much time it takes to solve the problem?

Performance analysis of an algorithm is performed by using the following measures.

- ❖ Space required to complete the task of that algorithm (**Space Complexity**).

It includes program space and data space.

- ❖ Time required to complete the task of that algorithm (**Time Complexity**)

Performance Measurement

Measuring an algorithm's efficiency is important because your choice of an **algorithm** for a given application often has a great impact. The analysis of **algorithms** is the area of computer science that provides tools for contrasting the **efficiency** of different methods of solution

Performance analysis estimates space and time complexity in advance

vs

while performance measurement measures the space and time taken in actual runs.

Arrays

“An array is collection of similar elements that are stored in sequential memory locations.”

- An ordinary variable can hold only one value at a time where as an array can store multiple values of same type.
- An *array* is a collection of variables of the same type that are referred to through a common name.
- A specific element in an array is accessed by an index. In C, all arrays consist of contiguous memory locations.

- The lowest address corresponds to the first element and the highest address to the last element.
- Arrays can have from one to several dimensions.
- The most common array is the *string*, which is simply an array of characters terminated by a null.

Single Dimensional Arrays

The general form for declaring a single-dimension array is
type var_name[size];

Like other variables, arrays must be explicitly declared so that the compiler can allocate space for them in memory. Here, *type* declares the base type of the array, which is the type of each element in the array, and *size* defines how many elements the array will hold. For example, to declare a 100- element array called **balance** of type **double**, use this statement:

```
double balance[100];
```

Example

```
int a[5];
```

a

a[0]	a[1]	a[2]	a[3]	a[4]
65516	65518	65520	65522	65524

- Here, **int** specifies the type of the variable, just as it does with ordinary variables and 'a' specifies the name of the variable.
- The **[5]** however is new. The number 5 tells how many elements of the type **int** will be in our array. This number is often called the 'dimension' of the array.
- The bracket ([]) tells the compiler that we are dealing with an array.
- a[0] refers to the first element in the array whereas the whole number 65516 is its address in memory.

Array Initialization

- So far we have used arrays that did not have any values in them to begin with.
- We managed to store values in them during program execution.
- Let us now see how to initialize an array while declaring it.

Example

```
int a[5] = { 2, 4, 6, 10, 12 };
or
int a[] = { 2, 4, 6, 10, 12 };
```


a

a[0]	a[1]	a[2]	a[3]	a[4]
2	4	6	8	10
65516	65518	65520	65522	65524

Array elements are referred to using subscript; the lowest subscript is always 0 and the highest subscript is (size -1). If you refer to an array element by using an out-of-range subscript, you will get an error. You can refer to any element as `a[0]`, `a[1]`, `a[2]`, etc

Thus, an array is a collection of similar elements. These similar elements could be all **ints**, or all **floats**, or all **chars**, etc. Usually, the array of characters is called a ‘string’, whereas an array of **ints** or **floats** is called simply an array. Remember that all elements of any given array must be of the same type. i.e. we cannot have an array of 10 numbers, of which 5 are **ints** and 5 are **floats**.

Example Program

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[5]={2,4,6,8,10};
clrscr();
printf("\nFirst element=%d",a[0]);
printf("\nFifth element=%d",a[4]);
getch();
}
```

Expected Output

First element=2
Fifth element=10

Dynamically Allocated Arrays

- In addition to dynamically allocating single values, we can also dynamically allocate arrays of variables.
- Unlike a fixed array, where the array size must be fixed at compile time, dynamically allocating an array allows us to choose an array length at runtime.
- To create a variable that will point to a dynamically allocated array, declare it as a pointer to the element type.
- For example, `int* a = NULL;` // pointer to an int, initially to nothing.
- A dynamically allocated array is declared as a pointer, and must not use the fixed array size declaration

Structures

Definition

- A structure contains a number of data types grouped together.
- These data types may or may not be of the same type.
- Unlike arrays which can store elements of same data type, a structure can hold data of different data types.

Declaring a Structure

The general form of a structure declaration statement is given below:

Syntax

```
struct <structure name>
{
    structure element 1 ;
    structure element 2 ;
    structure element 3 ;
    .....
    .....
};
```

Example

```
struct book
{
    char name ;
    float price ;
    int pages ;
};
```

Initialization of structures

Like primary variables and arrays, structure variables can also be initialized where they are declared. The format used is quite similar to that used to initiate arrays.

```
struct book
{
    char name[10] ;
    float price ;
    int pages ;
};
struct book b1 = { "Basic", 130.00, 550 } ;
struct book b2 = { "Physics", 150.80, 800 } ;
```

Example Program

```
#include<stdio.h>
#include<conio.h>

void main()
{
    struct book
    {
        char name[20];
        float price;
        int pages;
    };
    struct book b1={"Data Structures",275.50,450}; // Initialization of structure variable
    clrscr();
    printf("\nBook Name=%s",b1.name);
    printf("\nPrice=%.2f",b1.price);
    printf("\nPages=%d",b1.pages);
    getch();
}
```

Expected Output

Book Name=Data Structures

Price=275.50

Pages=450

Unions

- Like structure, a union can hold data belonging to different data types but it hold only one object at a time.
- In the structure each member has its own memory locations whereas, members of unions have same memory locations.
- The union requires bytes that are equal to the number of bytes required for the largest member.
- For example, if the union contains char, integer and float then the number of bytes reserved in the memory is 4 bytes (i.e. the size of float).
- Unlike structure members which can be initialized all at the same time, only one union member should be initialized at a time.

Syntax

The syntax of union is similar to the structure which is shown below

```
union <union_ name>
{
    union element 1 ;
    union element 2 ;
    union element 3 ;
    .....
    .....
```

```
};
```

Let us now observe the difference between union and structure by using the following program.

Example Program

//To show the difference between structure & union

```
struct student1          // structure
{
int rno;
char grade;
};
union student2          //union
{
int rno;
char grade;
};

void main()
{
struct student1 s={25,'A'}; // initialization of structure members at a time
union student2 u;
clrscr();

printf("\nRollno=%d",s.rno);
printf("\nGrade=%c",s.grade);

u.rno=30;          // initialization of union member 1
printf("\nRollno=%d",u.rno);
u.grade='B';       // initialization of union member 2
printf("\nGrade=%c",u.grade);

printf("\nSize of Structure=%d bytes",sizeof(s)); // displaying size of structure
printf("\nSize of Union=%d bytes",sizeof(u));    // displaying size of union
getch();
}
```

Example Program

```
Rollno=25
Grade=A
Rollno=50
Grade=B
Size of Structure=3 Bytes
Size of Union=2 Bytes
```

In the above, the size of structure is sum of the sizes of all its members i.e. int & char which is (2+1)=3 bytes whereas the size of union is the size of the member which belongs to largest data type i.e. int which is of 2 bytes.

Sorting

Motivation

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order.

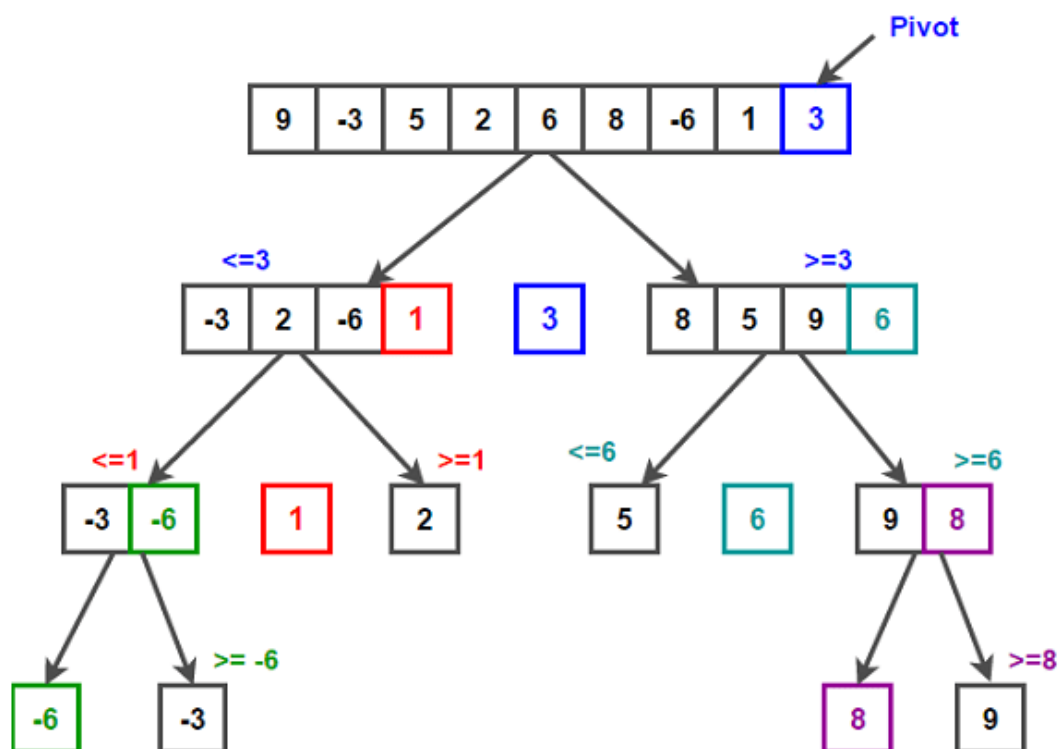
The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner. Sorting is also used to represent data in more readable formats. Following are some of the examples of sorting in real-life scenarios –

- **Telephone Directory** – The telephone directory stores the telephone numbers of people sorted by their names, so that the names can be searched easily.
- **Dictionary** – The dictionary stores words in an alphabetical order so that searching of any word becomes easy.

Quick Sort

Quick sort is a highly efficient sorting algorithm and is based on partitioning of array of data into smaller arrays. A large array is partitioned into two arrays one of which holds values smaller than the specified value, say pivot, based on which the partition is made and another array holds values greater than the pivot value.

Quicksort partitions an array and then calls itself recursively twice to sort the two resulting subarrays. This algorithm is quite efficient for large-sized data sets as its average and worst-case complexity are $O(n \log n)$



Merge Sort

Merge sort is a sorting technique based on divide and conquer technique.

It is one of the most respected algorithms.

Merge sort first divides the array into equal halves and then combines them in a sorted manner.

How Merge Sort Works?

To understand merge sort, we take an unsorted array as the following –



We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.



This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.



We further divide these arrays and we achieve atomic value which can no more be divided.

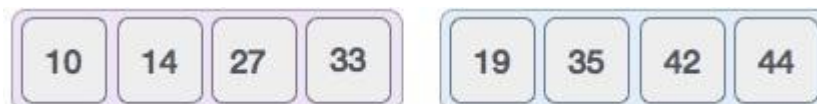


Now, we combine them in exactly the same manner as they were broken down. Please note the color codes given to these lists.

We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.



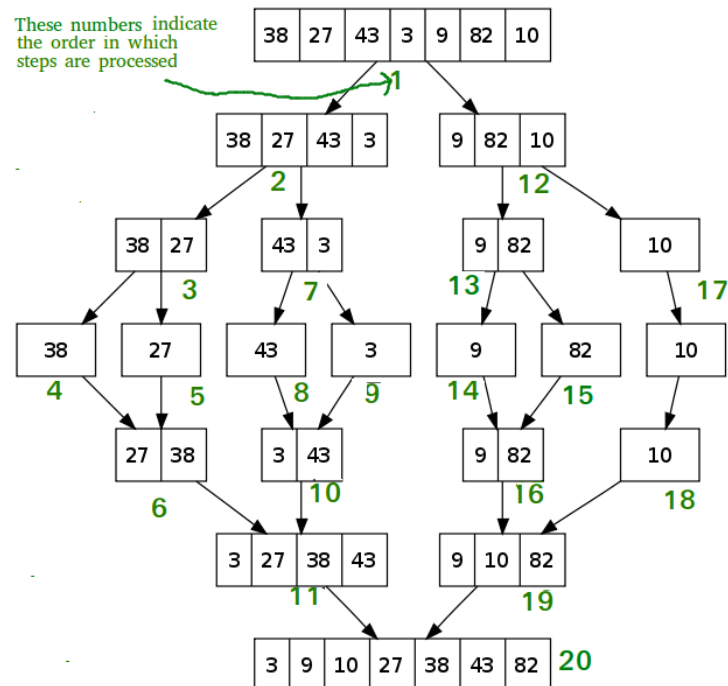
In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.



After the final merging, the list should look like this –



Example

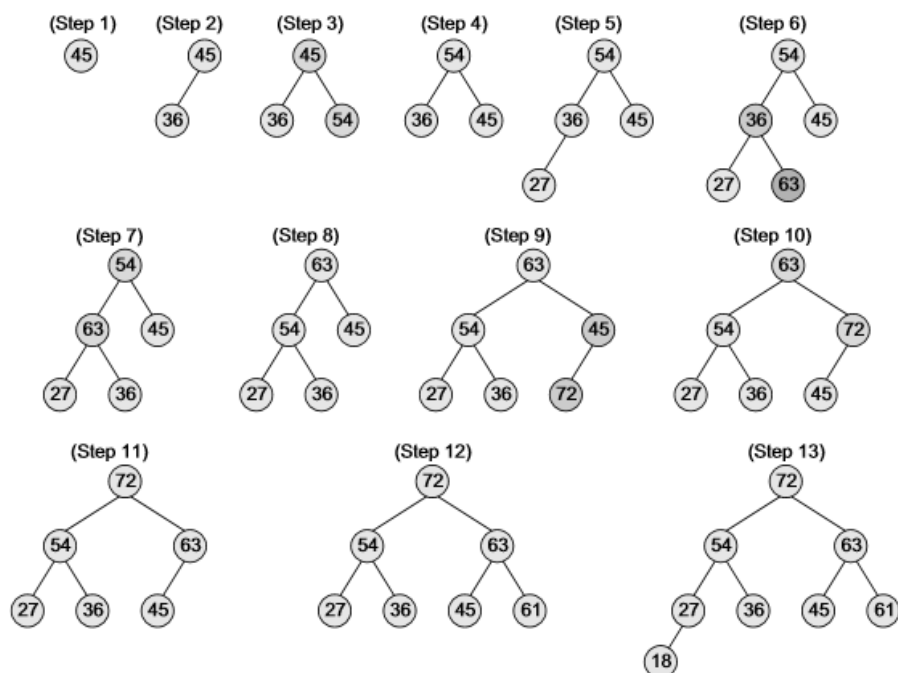


Heap Sort

Heaps can be used in sorting an array.

In max-heaps, maximum element will always be at the root. Heap Sort uses this property of heap to sort the array.

Example



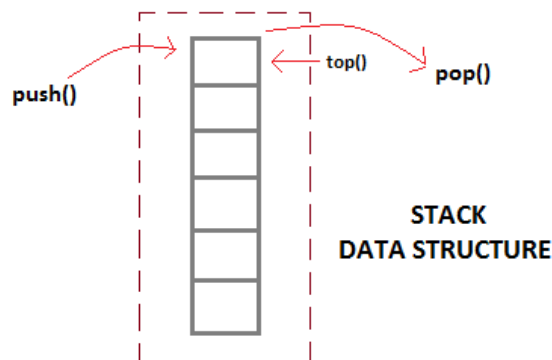
★★★★★★

UNIT-II

STACK, QUEUE AND LINKED LISTS

STACKS

Stack is an abstract data type with a bounded(predefined) capacity. It is a simple data structure that allows adding and removing elements in a particular order. Every time an element is added, it goes on the **top** of the stack and the only element that can be removed is the element that is at the top of the stack, just like a pile of objects.

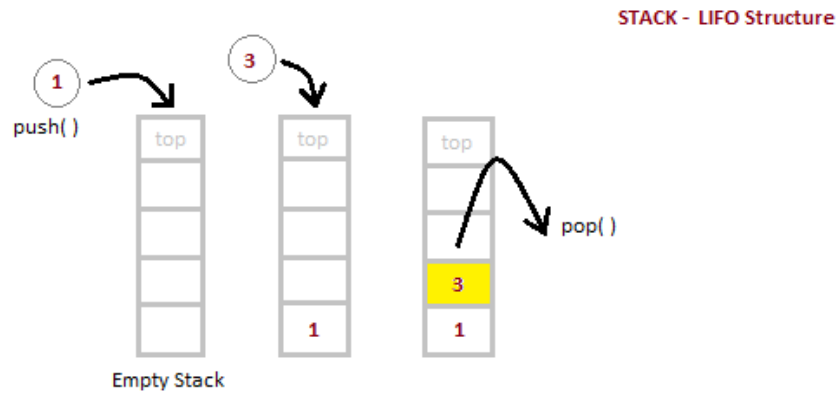


Basic features of Stack

1. Stack is an **ordered list** of **similar data type**.
 2. Stack is a **LIFO**(Last in First out) structure or we can say **FILO**(First in Last out).
 3. `push()` function is used to insert new elements into the Stack and `pop()` function is used to remove an element from the stack. Both insertion and removal are allowed at only one end of Stack called **Top**.
 4. Stack is said to be in **Overflow** state when it is completely full and is said to be in **Underflow** state if it is completely empty.
-

Implementation of Stack Data Structure

Stack can be easily implemented using an Array or a Linked List. Arrays are quick, but are limited in size and Linked List requires overhead to allocate, link, unlink, and deallocate, but is not limited in size. Here we will implement Stack using array.



In a Stack, all operations take place at the "top" of the stack. The "push" operation adds an item to the top of the Stack.
The "pop" operation removes the item on top of the stack.

Algorithm for PUSH operation

1. Check if the stack is **full** or not.
2. If the stack is full, then print error of overflow and exit the program.
3. If the stack is not full, then increment the top and add the element.

Algorithm for POP operation

1. Check if the stack is empty or not.
2. If the stack is empty, then print error of underflow and exit the program.
3. If the stack is not empty, then print the element at the top and decrement the top.

Below we have a simple C++ program implementing stack data structure while following the object oriented programming concepts.

Position of Top	Status of Stack
-1	Stack is Empty
0	Only one element in Stack
N-1	Stack is Full
N	Overflow state of Stack

Applications of Stack

The simplest application of a stack is to reverse a word. You push a given word to stack - letter by letter - and then pop letters from the stack.

There are other uses also like:

1. Parsing
2. Expression Conversion(Infix to Postfix, Postfix to Prefix etc)

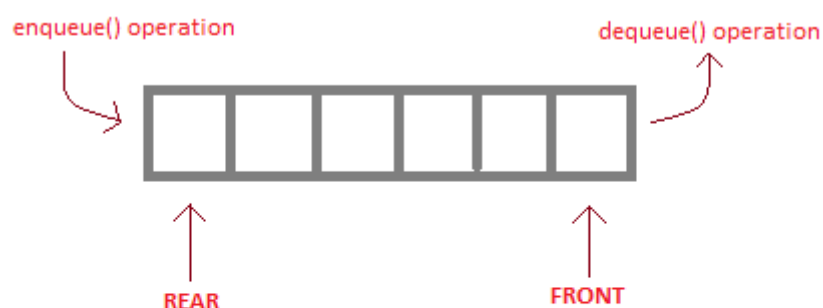
QUEUES

Queue is also an abstract data type or a linear data structure, just like stack data structure, in which the first element is inserted from one end called the **REAR**(also called **tail**), and the removal of existing element takes place from the other end called as **FRONT**(also called **head**).

This makes queue as **FIFO**(First in First Out) data structure, which means that element inserted first will be removed first.

Which is exactly how queue system works in real world. If you go to a ticket counter to buy movie tickets, and are first in the queue, then you will be the first one to get the tickets. Right? Same is the case with Queue data structure. Data inserted first, will leave the queue first.

The process to add an element into queue is called **Enqueue** and the process of removal of an element from queue is called **Dequeue**.



`enqueue( )` is the operation for adding an element into Queue.

`dequeue( )` is the operation for removing an element from Queue .

QUEUE DATA STRUCTURE

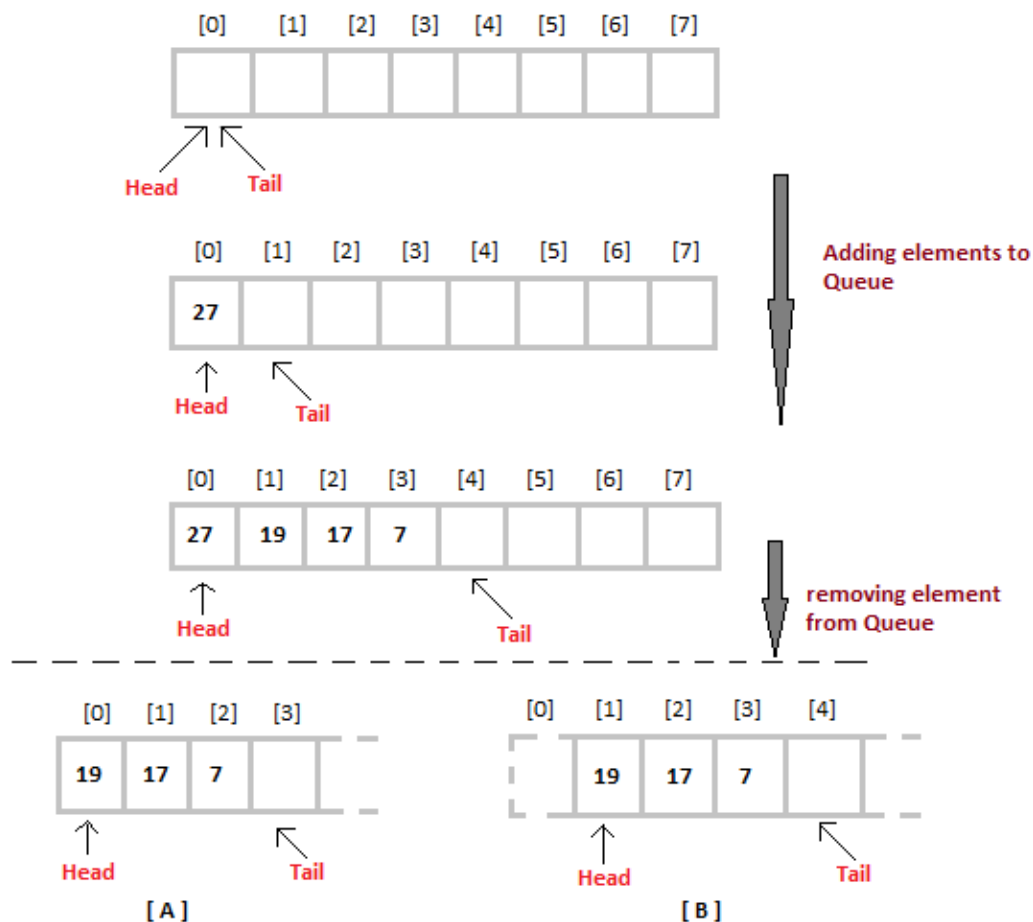
Basic features of Queue

1. Like stack, queue is also an ordered list of elements of similar data types.
2. Queue is a FIFO(First in First Out) structure.
3. Once a new element is inserted into the Queue, all the elements inserted before the new element in the queue must be removed, to remove the new element.
4. `peek( )` function is oftenly used to return the value of first element without dequeuing it.

Implementation of Queue Data Structure

Queue can be implemented using an Array, Stack or Linked List. The easiest way of implementing a queue is by using an Array.

Initially the **head**(FRONT) and the **tail**(REAR) of the queue points at the first index of the array (starting the index of array from 0). As we add elements to the queue, the **tail** keeps on moving ahead, always pointing to the position where the next element will be inserted, while the **head** remains at the first index.



When we remove an element from Queue, we can follow two possible approaches (mentioned [A] and [B] in above diagram). In [A] approach, we remove the element at **head** position, and then one by one shift all the other elements in forward position.

In approach [B] we remove the element from **head** position and then move **head** to the next position.

In approach [A] there is an **overhead of shifting the elements one position forward** every time we remove the first element.

In approach [B] there is no such overhead, but whenever we move head one position ahead, after removal of first element, the **size on Queue is reduced by one space** each time.

Algorithm for ENQUEUE operation

1. Check if the queue is full or not.
2. If the queue is full, then print overflow error and exit the program.
3. If the queue is not full, then increment the tail and add the element.

Algorithm for DEQUEUE operation

1. Check if the queue is empty or not.
2. If the queue is empty, then print underflow error and exit the program.
3. If the queue is not empty, then print the element at the head and increment the head.

Types of Queues

There are three types of queue:

- Circular Queue
- Priority Queue
- Deque

(i) Circular Queue

A circular queue is one in which the insertion of a element is done at the very first location if the queue if the last location of the queue is full. New element can be inserted if and only if the those location are empty For example, if we have a queue Q of say n elements , then after inserting an element at last location (i.e in the n-1th) location of the array the next element will be inserted at the very first location(i.e at 0th location) of the array.

A circular queue also have a front and rear to keep the track of the elements to be deleted and inserted and therefore to maintain the unique characteristic of the queue. Following are the assumption made:

- Front will always be pointing to the first element
- If front=rear the queue will be empty
- Each time a new element is inserted into the queue the rear is incremented by one
rear=rear+1
- Each time a new element is deleted from the queue the value of front is incremented by one, front=front+1

(ii) Priority Queue

Priority queue is a collection of elements such that each element has been assigned a priority and the order in which elements are deleted and processed comes from following rules:

- An element of higher priority is processed before any elements of lower priority
- Two elements with the same priority are processed according to the order in which they were added to the queue.

An example of priority queue in computer science occurs in timesharing system in which the processes of higher priority is executed before any process of lower priority.

There are two types of priority queue:

- **Ascending priority queue** : It is a collection of items in to which items can be inserted arbitrarily and from which only the smallest item can be removed.
- **Descending priority queue** : It is similar but allows deletion of only the largest item.

(iii) Double Ended Queue i.e Dequeue

It is also a homogeneous list of elements in which insertion and deletion operations are performed from both the ends. That is, we can insert elements from the rear end or from the front end. Hence, it is called Double-Ended Queue. There are two types of dequeues. These two types are due to the restrictions put to perform either the insertions or deletions only at one end.

- **Input restricted dequeues** : Input restricted dequeues allows insertions at only one end of the array or list but deletions allows at both ends.
- **Output restricted dequeues** : Output restricted dequeues allows deletions at only one end of the array or list but insertions allow at both ends.

Applications of Queue

Queue, as the name suggests is used whenever we need to manage any group of objects in an order in which the first one coming in, also gets out first while the others wait for their turn, like in the following scenarios:

1. Serving requests on a single shared resource, like a printer, CPU task scheduling etc.
2. In real life scenario, Call Center phone systems uses Queues to hold people calling them in an order, until a service representative is free.

Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive i.e First come first served.

Evaluation of Expressions

In any programming language, if we want to perform any calculation or to frame a condition etc., we use a set of symbols to perform the task. These set of symbols makes an expression.

An expression can be defined as follows...

An expression is a collection of operators and operands that represents a specific value.

In above definition, **operator** is a symbol which performs a particular task like arithmetic operation or logical operation or conditional operation etc.

Operands are the values on which the operators can perform the task. Here operand can be a direct value or variable or address of memory location.

Expression Types

Based on the operator position, expressions are divided into THREE types. They are as follows...

1. Infix Expression
2. Postfix Expression
3. Prefix Expression

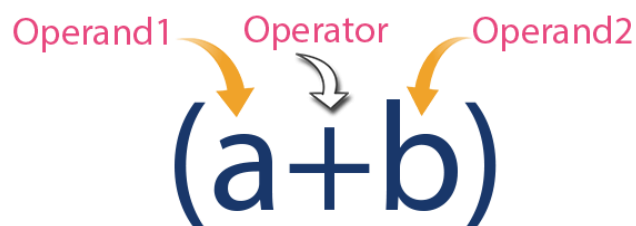
Infix Expression

In infix expression, operator is used in between the operands.

The general structure of an Infix expression is as follows...

Operand1 Operator Operand2

Example



Postfix Expression

In postfix expression, operator is used after operands. We can say that "**Operator follows the Operands**".

The general structure of Postfix expression is as follows...

Operand1 Operand2 Operator

Example



Prefix Expression

In prefix expression, operator is used before operands.

We can say that "**Operands follows the Operator**".

The general structure of Prefix expression is as follows...

Operator Operand1 Operand2

Example

Every expression can be represented using all the above three different types of expressions. And we can convert an expression from one form to another form like **Infix to Postfix**, **Infix to Prefix**, **Prefix to Postfix** and vice versa.

Linked Lists**Definition**

Linked List is a very commonly used linear data structure which consists of group of **nodes** in a sequence.

Each node holds its own **data** and the **address of the next node** hence forming a chain like structure.

Linked Lists are used to create trees and graphs.

**Advantages of Linked Lists**

- They are a dynamic in nature which allocates the memory when required.
- Insertion and deletion operations can be easily implemented.
- Stacks and queues can be easily executed.
- Linked List reduces the access time.

Disadvantages of Linked Lists

- The memory is wasted as pointers require extra memory for storage.
- No element can be accessed randomly; it has to access each node sequentially.
- Reverse Traversing is difficult in linked list.

Applications of Linked Lists

- Linked lists are used to implement stacks, queues, graphs, etc.
- Linked lists let you insert elements at the beginning and end of the list.
- In Linked Lists we don't need to know the size in advance.

Types of Linked Lists

There are 3 different implementations of Linked List available, they are:

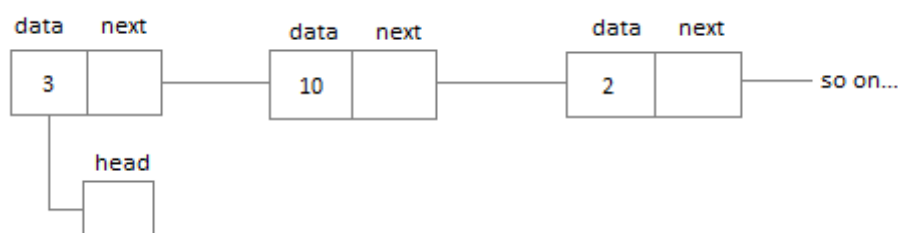
1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

Let's know more about them and how they are different from each other.

Singly Linked List

Singly linked lists contain nodes which have a **data** part as well as an **address part** i.e. next, which points to the next node in the sequence of nodes.

The operations we can perform on singly linked lists are **insertion**, **deletion** and **traversal**.



A linked list is a sequence of data structures, which are connected together via links.

Linked List is a sequence of links which contains items. Each link contains a connection to another link. Linked list is the second most-used data structure after array. Following are the important terms to understand the concept of Linked List.

- **Link** – Each link of a linked list can store a data called an element.
- **Next** – Each link of a linked list contains a link to the next link called Next.
- **Linked List** – A Linked List contains the connection link to the first link called First.

Linked List Representation

Linked list can be visualized as a chain of nodes, where every node points to the next node.

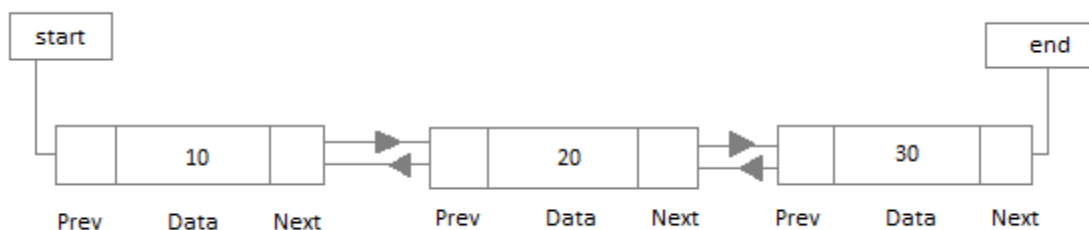


As per the above illustration, following are the important points to be considered.

- Linked List contains a link element called first.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Last link carries a link as null to mark the end of the list.

Doubly Linked List (Chains)

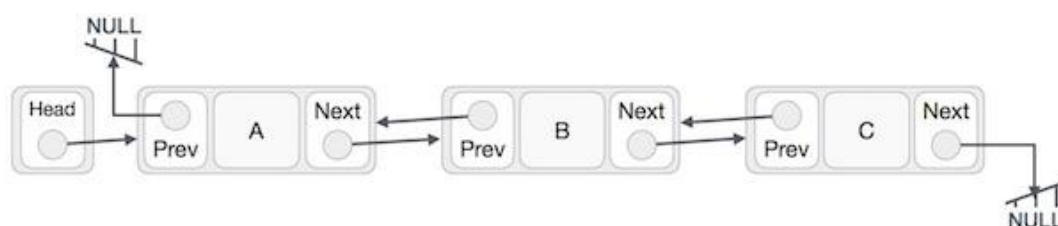
In a doubly linked list, each node contains a **data** part and two addresses, one for the **previous** node and one for the **next** node.



Doubly Linked List is a variation of Linked list in which navigation is possible in both ways, either forward and backward easily as compared to Single Linked List. Following are the important terms to understand the concept of doubly linked list.

- **Link** – Each link of a linked list can store a data called an element.
- **Next** – Each link of a linked list contains a link to the next link called Next.
- **Prev** – Each link of a linked list contains a link to the previous link called Prev.
- **Linked List** – A Linked List contains the connection link to the first link called First and to the last link called Last.

Representing Chains



As per the above illustration, following are the important points to be considered.

- Doubly Linked List contains a link element called first and last.
- Each link carries a data field(s) and two link fields called next and prev.

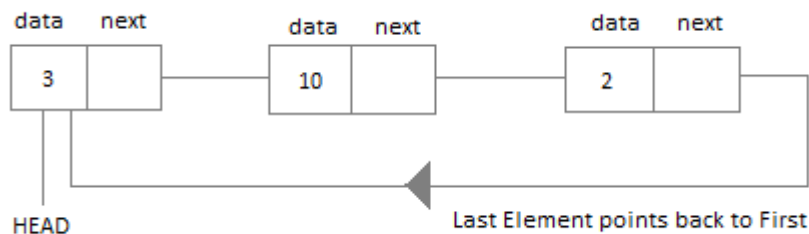
- Each link is linked with its next link using its next link.
- Each link is linked with its previous link using its previous link.
- The last link carries a link as null to mark the end of the list.

Circular Linked List

Circular Linked List is a variation of Linked list in which the first element points to the last element and the last element points to the first element.

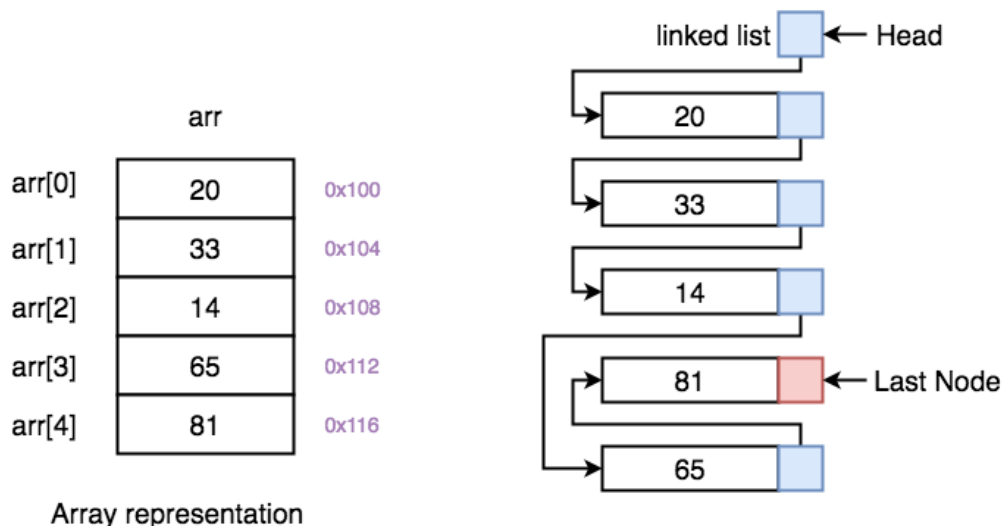
Both Singly Linked List and Doubly Linked List can be made into a circular linked list.

In circular linked list the last node of the list holds the address of the first node hence forming a circular chain.



Both Linked List and Array are used to store linear data of similar type, but an array consumes contiguous memory locations allocated at compile time, i.e. at the time of declaration of array, while for a linked list, memory is assigned as and when data is added to it, which means at runtime.

Below we have a pictorial representation showing how consecutive memory locations are allocated for array, while in case of linked list random memory locations are assigned to nodes, but each node is connected to its next node using **pointer**.



On the left, we have **Array** and on the right, we have **Linked List**.

Applications of Linked lists

Applications of linked list in computer science –

1. Implementation of stacks and queues
2. Implementation of graphs: Adjacency list representation of graphs is most popular which uses linked list to store adjacent vertices.
3. Dynamic memory allocation: We use linked list of free blocks.
4. Maintaining directory of names
5. Performing arithmetic operations on long integers
6. Manipulation of polynomials by storing constants in the node of linked list
7. representing sparse matrices

Applications of linked list in real world-

1. *Image viewer* – Previous and next images are linked, hence can be accessed by next and previous button.
2. *Previous and next page in web browser* – We can access previous and next url searched in web browser by pressing back and next button since, they are linked as linked list.
3. *Music Player* – Songs in music player are linked to previous and next song. you can play songs either from starting or ending of the list.

Applications of Circular Linked Lists:

1. Useful for implementation of queue. Unlike this implementation, we don't need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last.
2. Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.
3. Circular Doubly Linked Lists are used for implementation of advanced data structures like Fibonacci Heap.

★★★★★★

UNIT-III

TREES

What are trees?

- Tree is a hierarchical data structure which stores the information naturally in the form of hierarchy style.
- Tree is one of the most powerful and advanced data structures.
- It is a non-linear data structure compared to arrays, linked lists, stack and queue.
- It represents the nodes connected by edges.

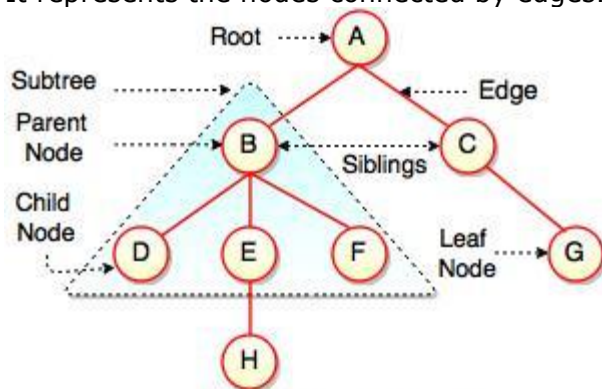


Fig. Structure of Tree

The above figure represents structure of a tree. Tree has 2 subtrees.

A is a parent of B and C.

B is called a child of A and also parent of D, E, F.

Tree is a collection of elements called Nodes, where each node can have arbitrary number of children.

Field	Description
Root	Root is a special node in a tree. The entire tree is referenced through it. It does not have a parent.
Parent Node	Parent node is an immediate predecessor of a node.
Child Node	All immediate successors of a node are its children.
Siblings	Nodes with the same parent are called Siblings.
Path	Path is a number of successive edges from source node to destination node.
Height of Node	Height of a node represents the number of edges on the longest path between that node and a leaf.
Height of Tree	Height of tree represents the height of its root node.

Depth of Node	Depth of a node represents the number of edges from the tree's root node to the node.
Degree of Node	Degree of a node represents a number of children of a node.
Edge	Edge is a connection between one node to another. It is a line between two nodes or a node and a leaf.

In the above figure, D, F, H, G are **leaves**. B and C are **siblings**. Each node excluding a root is connected by a direct edge from exactly one other node
parent → children.

Levels of a node

Levels of a node represents the number of connections between the node and the root. It represents generation of a node. If the root node is at level 0, its next node is at level 1, its grand child is at level 2 and so on. Levels of a node can be shown as follows:

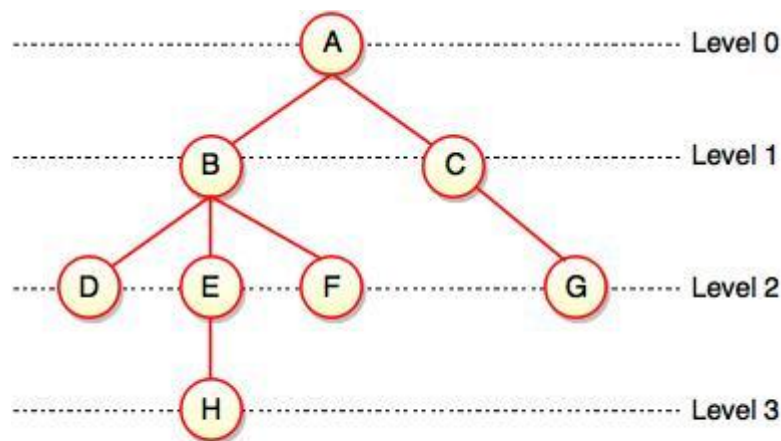


Fig. Levels of Tree

Note:

- If node has no children, it is called **Leaves** or **External Nodes**.
- Nodes which are not leaves, are called **Internal Nodes**. Internal nodes have at least one child.
- A tree can be empty with no nodes or a tree consists of one node called the **Root**.

Height of a Node

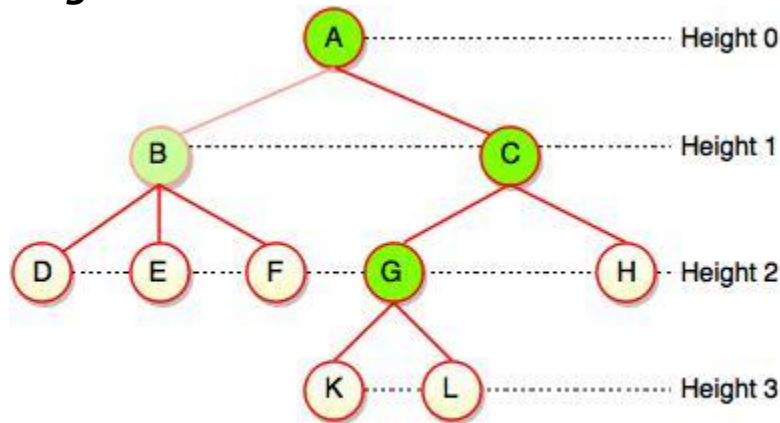


Fig. Height of a Node

As we studied, height of a node is a number of edges on the longest path between that node and a leaf. Each node has height.

In the above figure, A, B, C, D can have height. Leaf cannot have height as there will be no path starting from a leaf. Node A's height is the number of edges of the path to K not to D. And its height is 3.

Note:

- Height of a node defines the longest path from the node to a leaf.
- Path can only be downward.

Depth of a Node

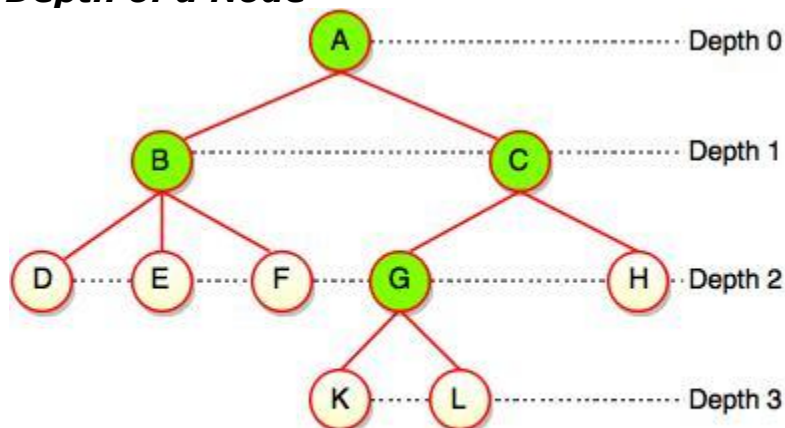


Fig. Depth of a Node

While talking about the height, it locates a node at bottom where for depth, it is located at top which is root level and therefore we call it depth of a node.

In the above figure, Node G's depth is 2. In depth of a node, we just count how many edges between the targeting node & the root and ignoring the directions.

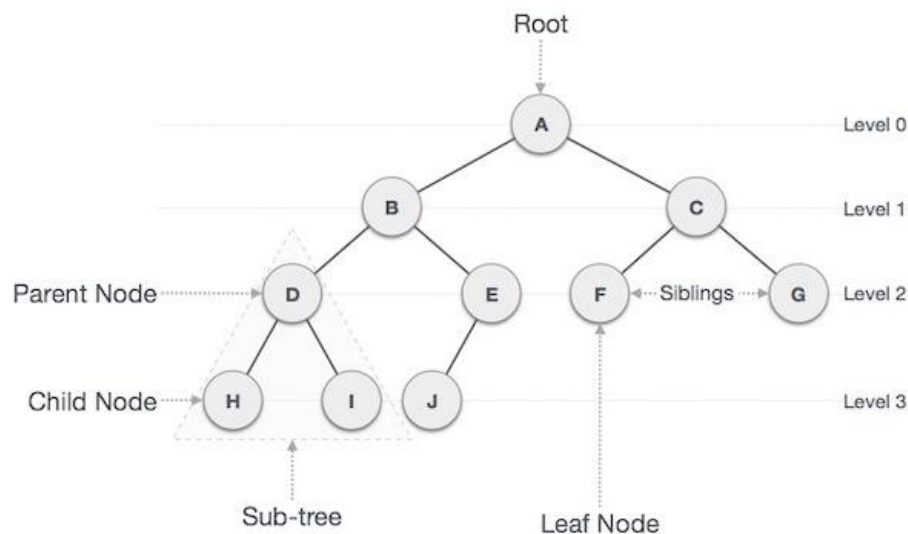
Note: Depth of the root is 0.

Advantages of Tree

- Tree reflects structural relationships in the data.
- It is used to represent hierarchies.
- It provides an efficient insertion and searching operations.
- Trees are flexible. It allows to move subtrees around with minimum effort.

Binary Tree

Binary Tree is a special data structure used for data storage purposes. A binary tree has a special condition that each node can have a maximum of two children. A binary tree has the benefits of both an ordered array and a linked list as search is as quick as in a sorted array and insertion or deletion operation are as fast as in linked list.



Binary Tree Traversals

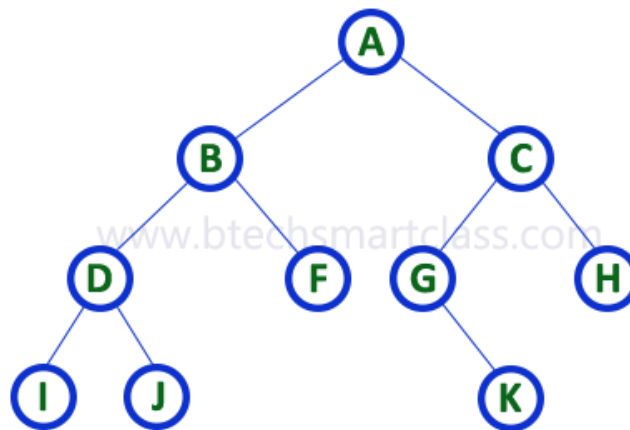
When we wanted to display a binary tree, we need to follow some order in which all the nodes of that binary tree must be displayed. In any binary tree, displaying order of nodes depends on the traversal method.

Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal.

There are three types of binary tree traversals.

1. In - Order Traversal
2. Pre - Order Traversal
3. Post - Order Traversal

Consider the following binary tree...



1. In - Order Traversal (leftChild - root - rightChild)

In In-Order traversal, the root node is visited between the left child and right child. In this traversal, the left child node is visited first, then the root node is visited and later we go for visiting the right child node. This in-order traversal is applicable for every root node of all subtrees in the tree. This is performed recursively for all nodes in the tree. In the above example of a binary tree, first we try to visit left child of root node 'A', but A's left child 'B' is a root node for left subtree. so we try to visit its (B's) left child 'D' and again D is a root for subtree with nodes D, I and J. So we try to visit its left child 'I' and it is the leftmost child. So first we visit 'I' then go for its root node 'D' and later we visit D's right child 'J'. With this we have completed the left part of node B. Then visit 'B' and next B's right child 'F' is visited. With this we have completed left part of node A. Then visit root node 'A'. With this we have completed left and root parts of node A. Then we go for the right part of the node A. In right of A again there is a subtree with root C. So go for left child of C and again it is a subtree with root G. But G does not have left part so we visit 'G' and then visit G's right child K. With this we have completed the left part of node C. Then visit root node 'C' and next visit C's right child 'H' which is the rightmost child in the tree. So we stop the process.

That means here we have visited in the order of **I - D - J - B - F - A - G - K - C - H** using In-Order Traversal.

In-Order Traversal for above example of binary tree is

I - D - J - B - F - A - G - K - C - H

2. Pre - Order Traversal (root - leftChild - rightChild)

In Pre-Order traversal, the root node is visited before the left child and right child nodes. In this traversal, the root node is visited first, then its left child and later its right child. This pre-order traversal is applicable for every root node of all subtrees in the tree. In the above example of binary tree, first we visit root node 'A' then visit its left child 'B' which is a root for D and F. So we visit B's left child 'D' and again D is a root for I

and J. So we visit D's left child 'I' which is the leftmost child. So next we go for visiting D's right child 'J'. With this we have completed root, left and right parts of node D and root, left parts of node B. Next visit B's right child 'F'. With this we have completed root and left parts of node A. So we go for A's right child 'C' which is a root node for G and H. After visiting C, we go for its left child 'G' which is a root for node K. So next we visit left of G, but it does not have left child so we go for G's right child 'K'. With this, we have completed node C's root and left parts. Next visit C's right child 'H' which is the rightmost child in the tree.

So we stop the process.

That means here we have visited in the order of **A-B-D-I-J-F-C-G-K-H** using Pre-Order Traversal.

Pre-Order Traversal for above example binary tree is

A - B - D - I - J - F - C - G - K - H

3. Post - Order Traversal (leftChild - rightChild - root)

In Post-Order traversal, the root node is visited after left child and right child. In this traversal, left child node is visited first, then its right child and then its root node. This is recursively performed until the right most node is visited.

Here we have visited in the order of **I - J - D - F - B - K - G - H - C - A** using Post-Order Traversal.

Post-Order Traversal for above example binary tree is

I - J - D - F - B - K - G - H - C - A

Binary Search Tree Operations

Following are the operations performed on binary search tree:

1. Insert Operation

- Insert operation is performed with $O(\log n)$ time complexity in a binary search tree.
- Insert operation starts from the root node. It is used whenever an element is to be inserted.

The following algorithm shows the insert operation in binary search tree:

Step 1: Create a new node with a value and set its left and right to NULL.

Step 2: Check whether the tree is empty or not.

Step 3: If the tree is empty, set the root to a new node.

Step 4: If the tree is not empty, check whether a value of new node is smaller or larger than the node (here it is a root node).

Step 5: If a new node is smaller than or equal to the node, move to its left child.

Step 6: If a new node is larger than the node, move to its right child.

Step 7: Repeat the process until we reach to a leaf node.

2. Search Operation

- Search operation is performed with $O(\log n)$ time complexity in a binary search tree.
- This operation starts from the root node. It is used whenever an element is to be searched.

The following algorithm shows the search operation in binary search tree:

Step 1: Read the element from the user .

Step 2: Compare this element with the value of root node in a tree.

Step 3: If element and value are matching, display "Node is Found" and terminate the function.

Step 4: If element and value are not matching, check whether an element is smaller or larger than a node value.

Step 5: If an element is smaller, continue the search operation in left subtree.

Step 6: If an element is larger, continue the search operation in right subtree.

Step 7: Repeat the same process until we found the exact element.

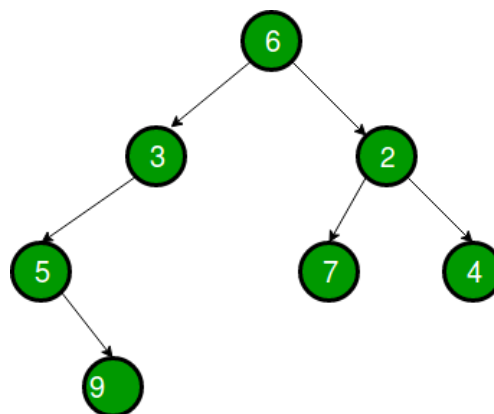
Step 8: If an element with search value is found, display "Element is found" and terminate the function.

Step 9: If we reach to a leaf node and the search value is not match to a leaf node, display "Element is not found" and terminate the function.

Counting Binary Trees

Given A binary Tree, how do you count all the full nodes (Nodes which have both children as not NULL).

Note leaves should not be touched as they have both children as NULL.



Nodes 2 and 6 are full nodes has both child's. So count of full nodes in the above tree is 2

Types of Binary Trees

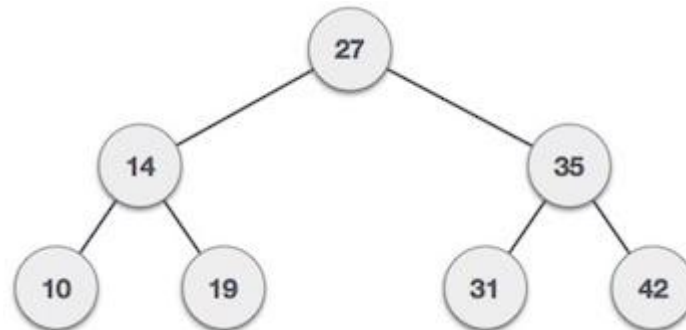
The following are the various types of Binary Trees.

- Binary Search Trees
- Heap Trees
- Height Balanced Trees

- (iv) B-Trees
- (v) Red Black Trees

(i) Binary Search Tree Representation

Binary Search tree exhibits a special behavior. A node's left child must have a value less than its parent's value and the node's right child must have a value greater than its parent value.



We're going to implement tree using node object and connecting them through references.

The basic operations that can be performed on a binary search tree data structure, are the following –

- **Insert** – Inserts an element in a tree/create a tree.
- **Search** – Searches an element in a tree.
- **Preorder Traversal** – Traverses a tree in a pre-order manner.
- **Inorder Traversal** – Traverses a tree in an in-order manner.
- **Postorder Traversal** – Traverses a tree in a post-order manner.

Optimal Binary Search Trees

In computer science, an **optimal binary search tree (Optimal BST)**, sometimes called a **weight-balanced binary tree**, is a binary search tree which provides the smallest possible search time (or expected search time) for a given sequence of accesses (or access probabilities). Optimal BSTs are generally divided into two types: static and dynamic.

In the **static optimality** problem, the tree cannot be modified after it has been constructed. In this case, there exists some particular layout of the nodes of the tree which provides the smallest expected search time for the given access probabilities. Various algorithms exist to construct or approximate the statically optimal tree given the information on the access probabilities of the elements.

In the **dynamic optimality** problem, the tree can be modified at any time, typically by permitting tree rotations. The tree is considered to have a cursor starting at the root which it can move or use to perform modifications. In this case, there exists some minimal-cost sequence of these operations which causes the cursor to visit every node in the target access sequence in order.

(ii) Heap Trees

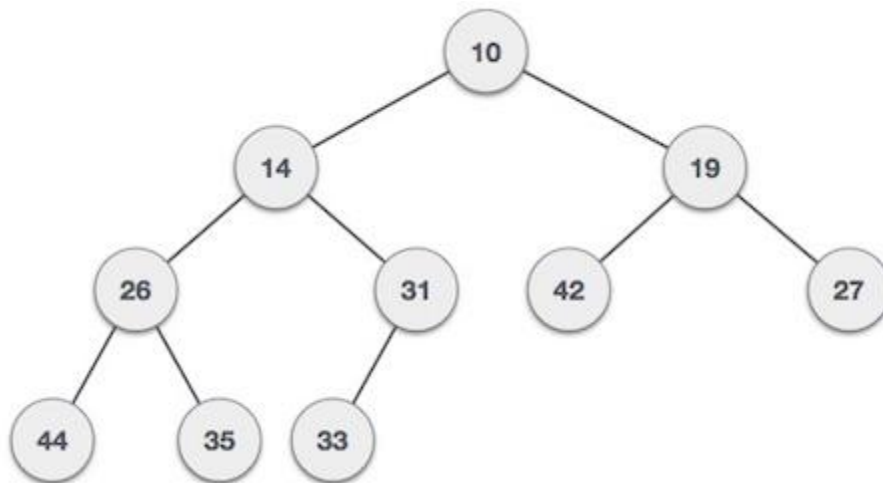
Heap is a special case of balanced binary tree data structure where the root-node key is compared with its children and arranged accordingly. If α has child node β then –

$$\text{key}(\alpha) \geq \text{key}(\beta)$$

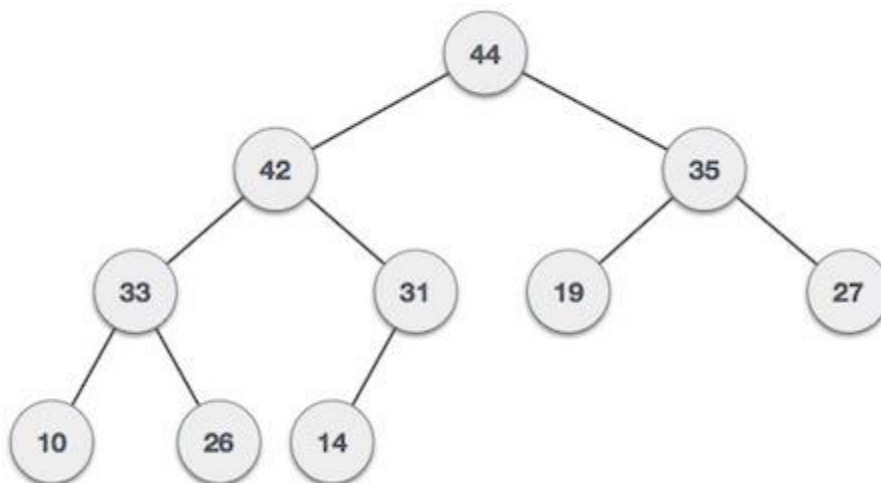
As the value of parent is greater than that of child, this property generates **Max Heap**. Based on this criteria, a heap can be of two types –

For Input → 35 33 42 10 14 19 27 44 26 31

Min-Heap – Where the value of the root node is less than or equal to either of its children.



Max-Heap – Where the value of the root node is greater than or equal to either of its children.



Both trees are constructed using the same input and order of arrival.

Max Heap Construction Algorithm

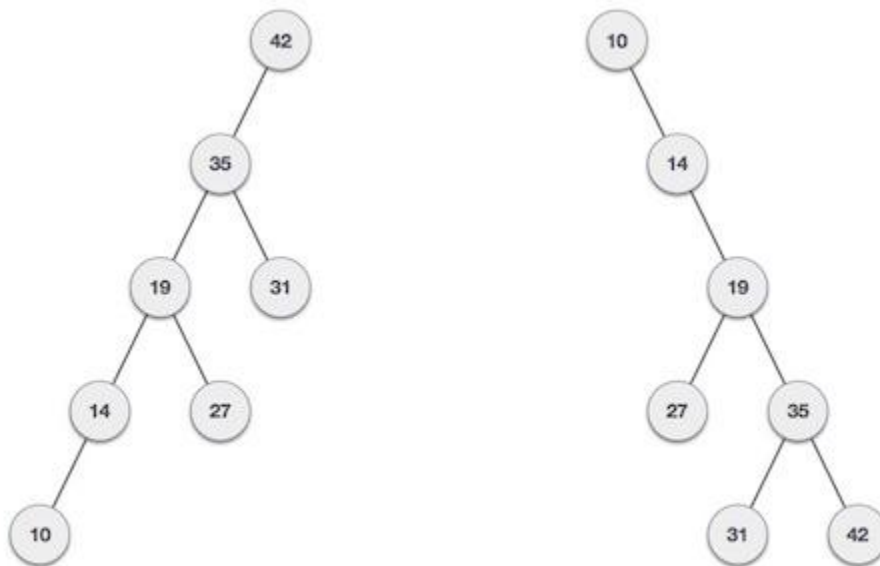
We shall use the same example to demonstrate how a Max Heap is created. The procedure to create Min Heap is similar but we go for min values instead of max values.

We are going to derive an algorithm for max heap by inserting one element at a time. At any point of time, heap must maintain its property. While insertion, we also assume that we are inserting a node in an already heapified tree.

- Step 1** – Create a new node at the end of heap.
Step 2 – Assign new value to the node.
Step 3 – Compare the value of this child node with its parent.
Step 4 – If value of parent is less than child, then swap them.
Step 5 – Repeat step 3 & 4 until Heap property holds.

(iii) Height Balanced Trees (AVL Trees)

What if the input to binary search tree comes in a sorted (ascending or descending) manner?
 It will then look like this –



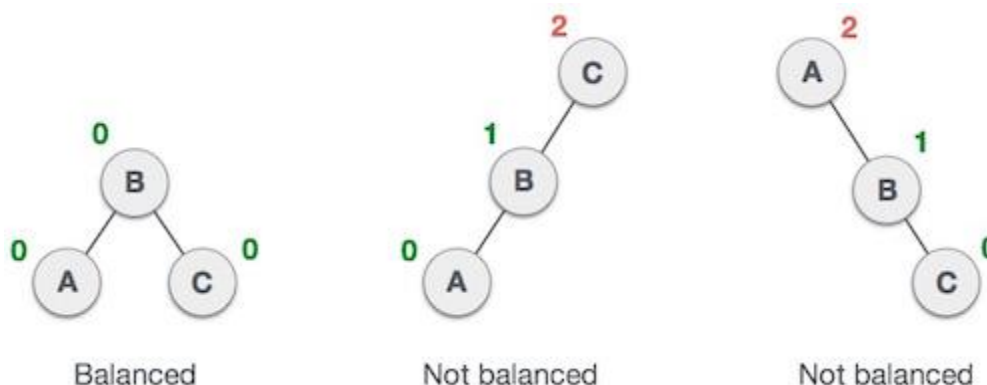
If input 'appears' non-increasing manner

If input 'appears' in non-decreasing manner

It is observed that BST's worst-case performance is closest to linear search algorithms, that is $O(n)$. In real-time data, we cannot predict data pattern and their frequencies. So, a need arises to balance out the existing BST.

Named after their inventor **Adelson, Velski & Landis**, **AVL trees** are height balancing binary search tree. AVL tree checks the height of the left and the right sub-trees and assures that the difference is not more than 1. This difference is called the **Balance Factor**.

Here we see that the first tree is balanced and the next two trees are not balanced –



In the second tree, the left subtree of **C** has height 2 and the right subtree has height 0, so the difference is 2. In the third tree, the right subtree of **A** has height 2 and the left is missing, so it is 0, and the difference is 2 again. AVL tree permits difference (balance factor) to be only 1.

BalanceFactor = height(left-subtree) – height(right-subtree)

If the difference in the height of left and right sub-trees is more than 1, the tree is balanced using some rotation techniques.

AVL Rotations

To balance itself, an AVL tree may perform the following four kinds of rotations –

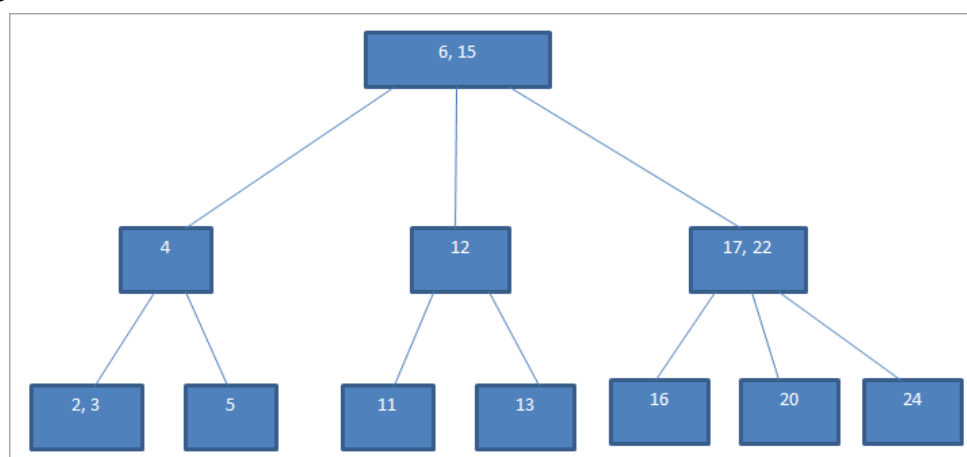
- Left rotation
- Right rotation
- Left-Right rotation
- Right-Left rotation

The first two rotations are single rotations and the next two rotations are double rotations. To have an unbalanced tree, we at least need a tree of height 2.

(iv) B Trees

B-Tree is a self-balancing search tree. In most of the other self-balancing search trees (like AVL and Red-Black Trees), it is assumed that everything is in main memory. To understand the use of B-Trees, we must think of the huge amount of data that cannot fit in main memory. When the number of keys is high, the data is read from disk in the form of blocks. Disk access time is very high compared to main memory access time. The main idea of using B-Trees is to reduce the number of disk accesses. Most of the tree operations (search, insert, delete, max, min, ..etc) require $O(h)$ disk accesses where h is the height of the tree. B-tree is a fat tree. The height of B-Trees is kept low by putting maximum possible keys in a B-Tree node. Generally, a B-Tree node size is kept equal to the disk block size. Since h is low for B-Tree, total disk accesses for most of the operations are reduced significantly compared to balanced Binary Search Trees like AVL Tree, Red-Black Tree, ..etc.

Example



Properties of B-Tree

- All leaves of B-tree are at the same level.
- A B-tree of order m can have at most $m-1$ keys and m children.
- Every node in B-tree has at most m children.
- Root node must have at least two nodes.
- Every node except the root node and the leaf node contain $m/2$ children

Applications of B Trees

- B trees are used to index the data especially in large databases as access to data stored in large databases on disks is very time-consuming.
- Searching of data in larger unsorted data sets takes a lot of time but this can be improved significantly with indexing using B tree.

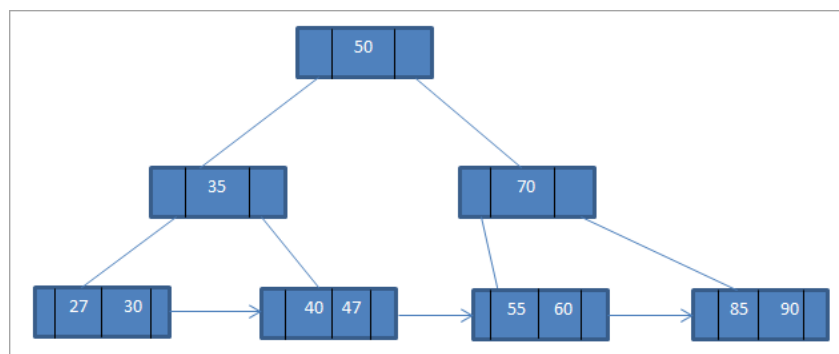
B+ Trees

B+ tree is an extension of the B tree. The difference in B+ tree and B tree is that in B tree the keys and records can be stored as internal as well as leaf nodes whereas in B+ trees, the records are stored as leaf nodes and the keys are stored only in internal nodes.

The records are linked to each other in a linked list fashion. This arrangement makes the searches of B+ trees faster and efficient. Internal nodes of the B+ tree are called index nodes.

The B+ trees have two orders i.e. one for internal nodes and other for leaf or external nodes.

Example



As B+ tree is an extension of B-tree, the basic operations that we discussed under the topic B-tree still holds.

While inserting as well as deleting, we should maintain the basic properties of B+ Trees intact. However, deletion operation in the B+ tree is comparatively easier as the data is stored only in the leaf nodes and it will be deleted from the leaf nodes always.

Advantages of B+ Trees

- We can fetch records in an equal number of disk accesses.
- Compared to the B tree, the height of the B+ tree is less and remains balanced.
- We use keys for indexing.
- Data in the B+ tree can be accessed sequentially or directly as the leaf nodes are arranged in a linked list.

- Search is faster as data is stored in leaf nodes only and as a linked list.

Difference Between B-Tree And B+ Tree

B-Tree	B+ Tree
Data is stored in leaf nodes as well as internal nodes.	Data is stored only in leaf nodes.
Searching is a bit slower as data is stored in internal as well as leaf nodes.	Searching is faster as the data is stored only in the leaf nodes.
No redundant search keys are present.	Redundant search keys may be present.
Deletion operation is complex.	Deletion operation is easy as data can be directly deleted from the leaf nodes.
Leaf nodes cannot be linked together.	Leaf nodes are linked together to form a linked list.

(v) Red Black Trees

Red - Black Tree is another variant of Binary Search Tree in which every node is colored either RED or BLACK. We can define a Red Black Tree as follows...

Red Black Tree is a Binary Search Tree in which every node is colored either RED or BLACK.

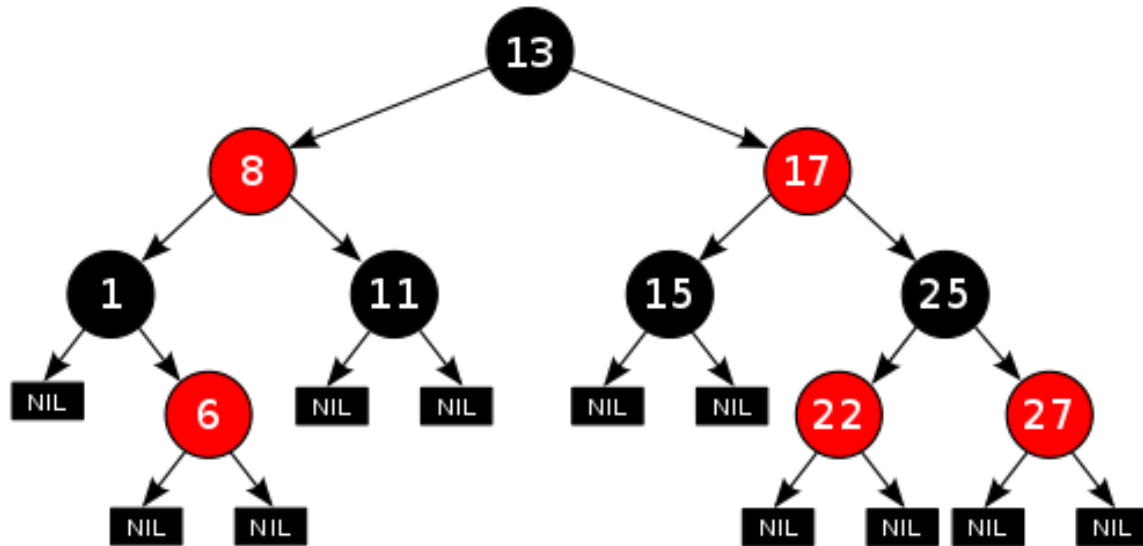
In Red Black Tree, the color of a node is decided based on the properties of Red Black Tree. Every Red Black Tree has the following properties.

Properties of Red Black Tree

- **Property #1:** Red - Black Tree must be a Binary Search Tree.
- **Property #2:** The ROOT node must be colored BLACK.
- **Property #3:** The children of Red colored node must be colored BLACK. (There should not be two consecutive RED nodes).
- **Property #4:** In all the paths of the tree, there should be same number of BLACK colored nodes.
- **Property #5:** Every new node must be inserted with RED color.
- **Property #6:** Every leaf (e.i. NULL node) must be colored BLACK.

Example

Following is a Red Black Tree which is created by inserting numbers from 1 to 9.



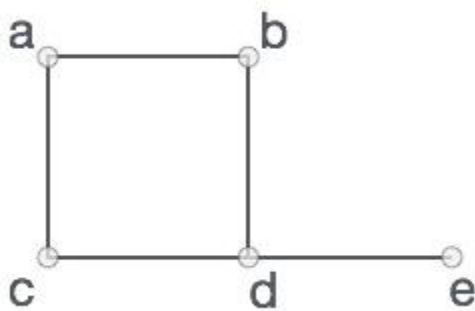
UNIT-IV

GRAPHS AND HASHING

The Graph Abstract Data Type

A graph is a pictorial representation of a set of objects where some pairs of objects are connected by links. The interconnected objects are represented by points termed as **vertices**, and the links that connect the vertices are called **edges**.

Formally, a graph is a pair of sets **(V, E)**, where **V** is the set of vertices and **E** is the set of edges, connecting the pairs of vertices. Take a look at the following graph.



In the above graph,

$$V = \{a, b, c, d, e\}$$

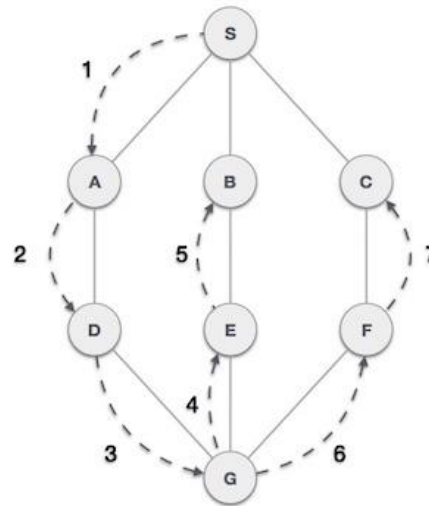
$$E = \{ab, ac, bd, cd, de\}$$

Elementary Graph Operations

Following are basic primary operations of a Graph –

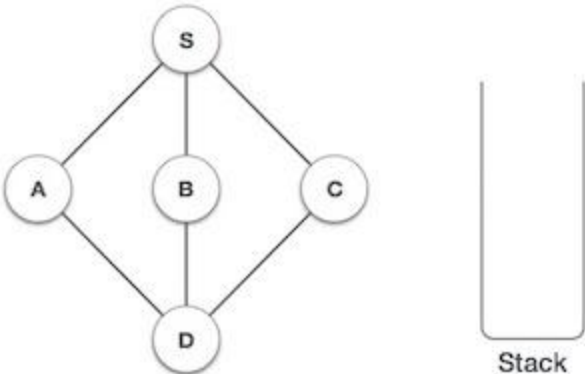
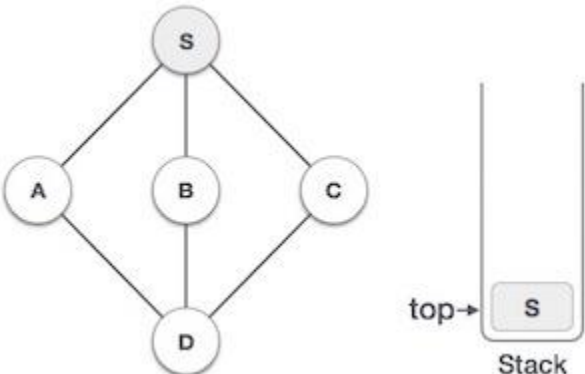
- **Add Vertex** – Adds a vertex to the graph.
- **Add Edge** – Adds an edge between the two vertices of the graph.
- **Display Vertex** – Displays a vertex of the graph.

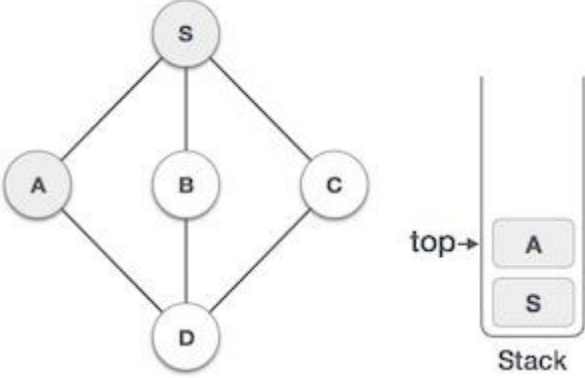
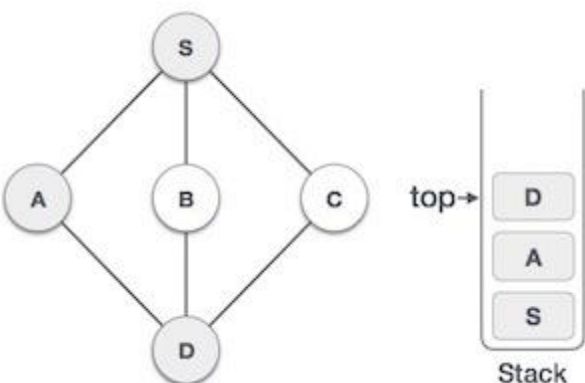
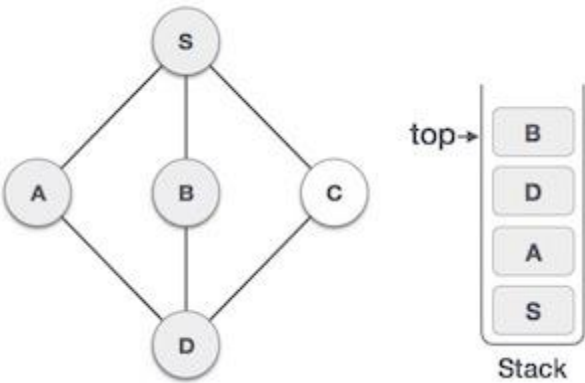
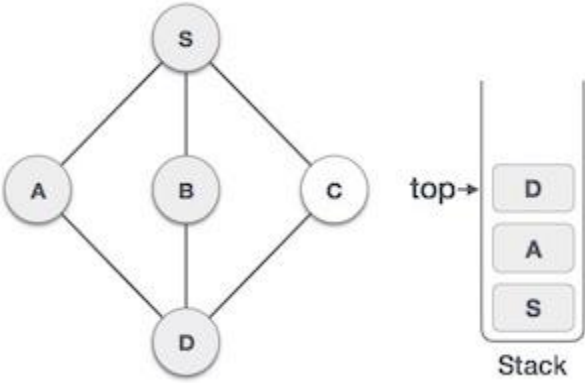
Depth First Search (DFS) algorithm traverses a graph in a depthward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

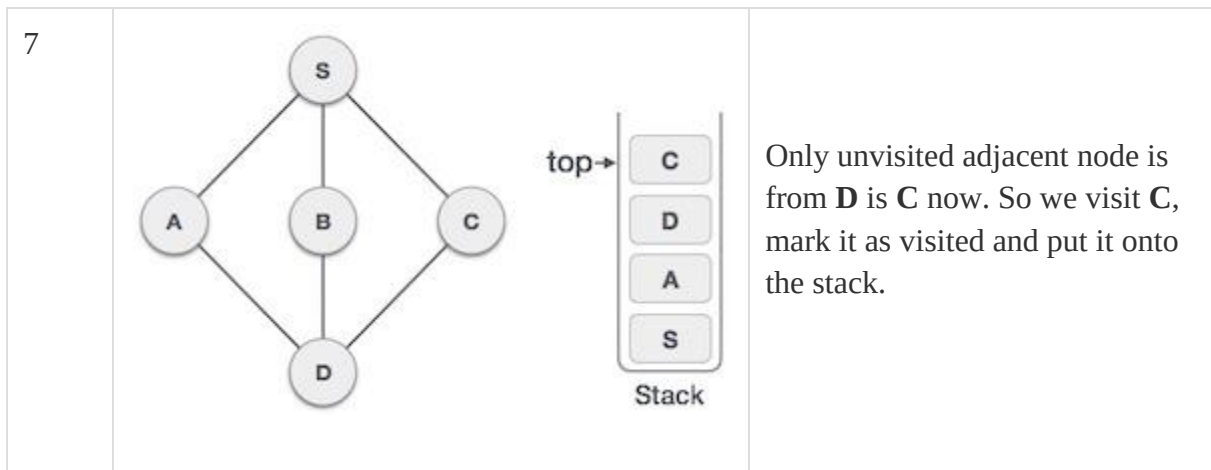


As in the example given above, DFS algorithm traverses from S to A to D to G to E to B first, then to F and lastly to C. It employs the following rules.

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
- **Rule 2** – If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
- **Rule 3** – Repeat Rule 1 and Rule 2 until the stack is empty.

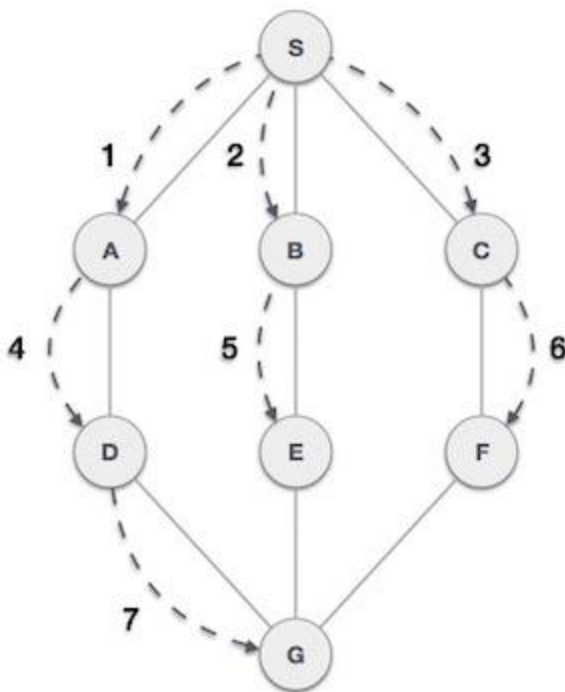
Step	Traversal	Description
1		Initialize the stack.
2		Mark S as visited and put it onto the stack. Explore any unvisited adjacent node from S. We have three nodes and we can pick any of them. For this example, we shall take the node in an alphabetical order.

3		Mark A as visited and put it onto the stack. Explore any unvisited adjacent node from A. Both S and D are adjacent to A but we are concerned for unvisited nodes only.
4		Visit D and mark it as visited and put onto the stack. Here, we have B and C nodes, which are adjacent to D and both are unvisited. However, we shall again choose in an alphabetical order.
5		We choose B, mark it as visited and put onto the stack. Here B does not have any unvisited adjacent node. So, we pop B from the stack.
6		We check the stack top for return to the previous node and check if it has any unvisited nodes. Here, we find D to be on the top of the stack.



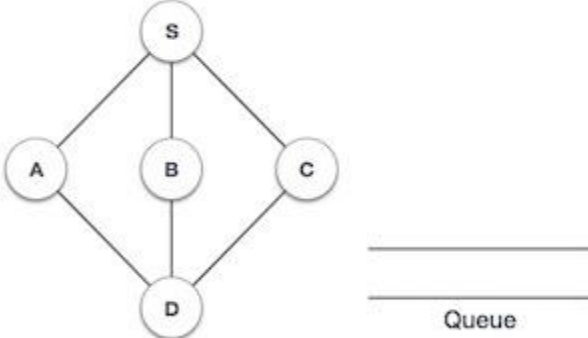
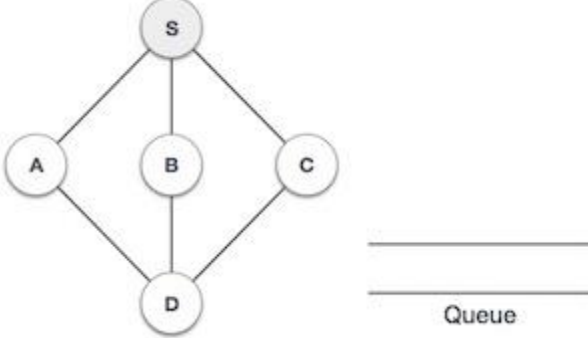
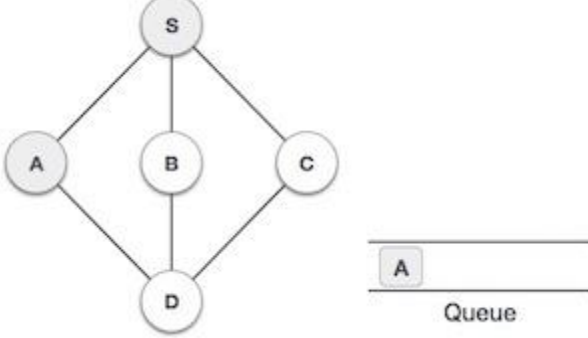
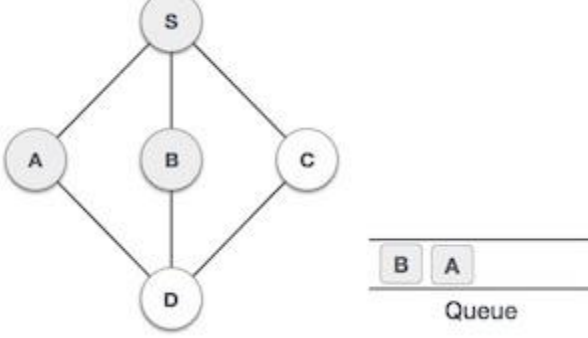
As **C** does not have any unvisited adjacent node so we keep popping the stack until we find a node that has an unvisited adjacent node. In this case, there's none and we keep popping until the stack is empty.

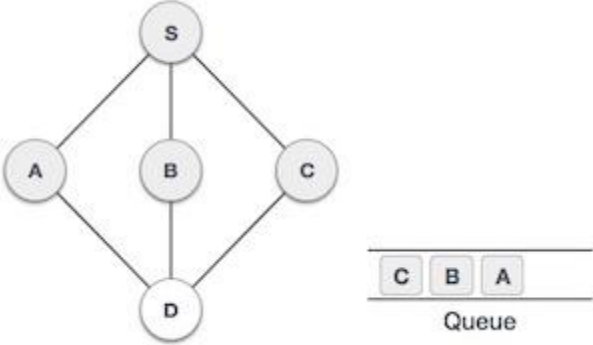
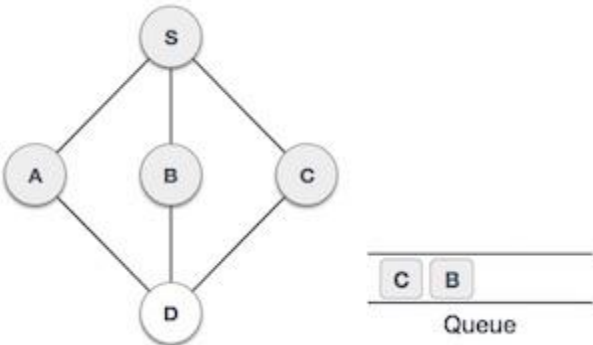
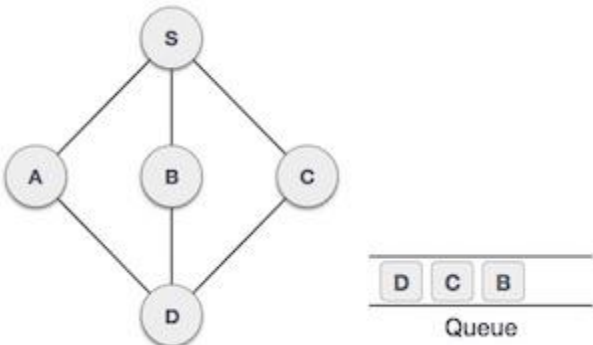
Breadth First Search (BFS) algorithm traverses a graph in a breadthward motion and uses a queue to remember to get the next vertex to start a search, when a dead end occurs in any iteration.



As in the example given above, BFS algorithm traverses from A to B to E to F first then to C and G lastly to D. It employs the following rules.

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.
- **Rule 2** – If no adjacent vertex is found, remove the first vertex from the queue.
- **Rule 3** – Repeat Rule 1 and Rule 2 until the queue is empty.

Step	Traversal	Description
1		Initialize the queue.
2		We start from visiting S (starting node), and mark it as visited.
3		We then see an unvisited adjacent node from S . In this example, we have three nodes but alphabetically we choose A , mark it as visited and enqueue it.
4		Next, the unvisited adjacent node from S is B . We mark it as visited and enqueue it.

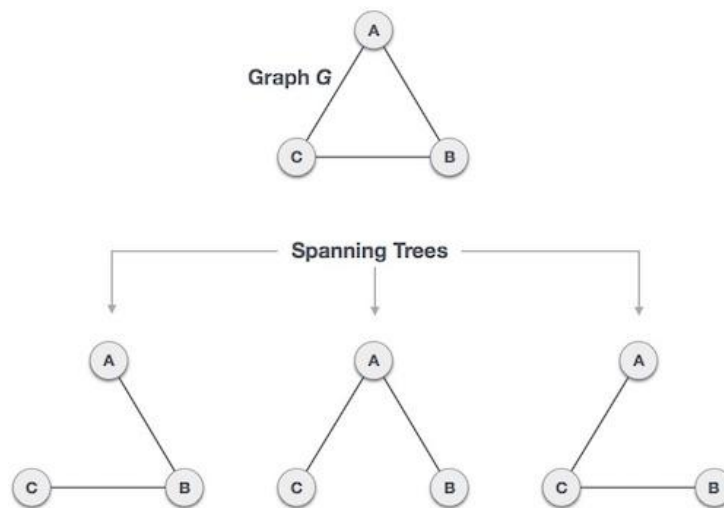
5		Next, the unvisited adjacent node from S is C. We mark it as visited and enqueue it.
6		Now, S is left with no unvisited adjacent nodes. So, we dequeue and find A.
7		From A we have D as unvisited adjacent node. We mark it as visited and enqueue it.

At this stage, we are left with no unmarked (unvisited) nodes. But as per the algorithm we keep on dequeuing in order to get all unvisited nodes. When the queue gets emptied, the program is over.

Spanning Trees

A spanning tree is a subset of Graph G , which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..

By this definition, we can draw a conclusion that every connected and undirected Graph G has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.



We found three spanning trees off one complete graph. A complete undirected graph can have maximum n^{n-2} number of spanning trees, where n is the number of nodes. In the above addressed example, n is 3, hence $3^{3-2} = 3$ spanning trees are possible.

Minimum Cost Spanning Trees

In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph. In real-world situations, this weight can be measured as distance, congestion, traffic load or any arbitrary value denoted to the edges.

Minimum Spanning-Tree Algorithm

We shall learn about two most important spanning tree algorithms here

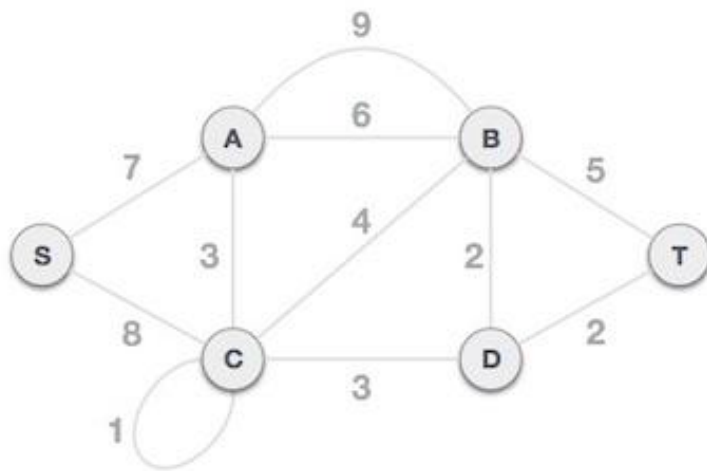
- Kruskal's Algorithm
- Prim's Algorithm

Both are greedy algorithms.

(1) Kruskal's Algorithm

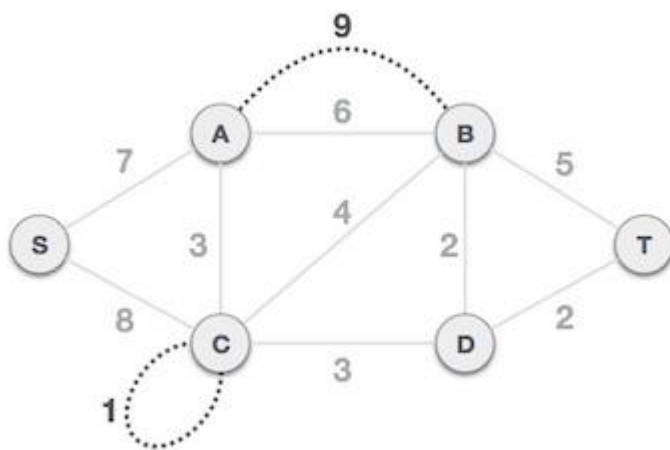
Kruskal's algorithm to find the minimum cost spanning tree uses the greedy approach. This algorithm treats the graph as a forest and every node it has as an individual tree. A tree connects to another only and only if, it has the least cost among all available options and does not violate MST properties.

To understand Kruskal's algorithm let us consider the following example –

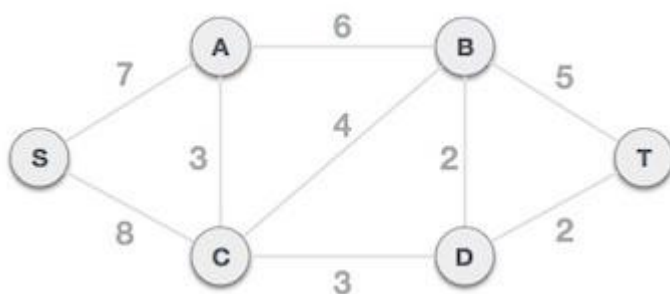


Step 1 - Remove all loops and Parallel Edges

Remove all loops and parallel edges from the given graph.



In case of parallel edges, keep the one which has the least cost associated and remove all others.



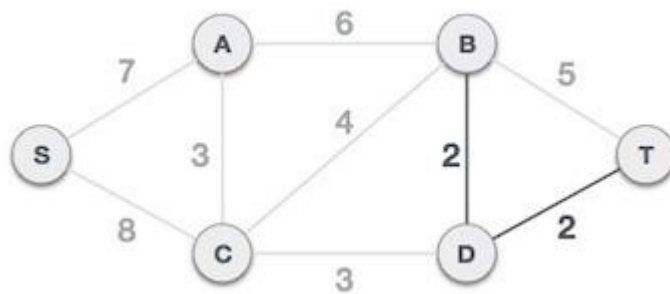
Step 2 - Arrange all edges in their increasing order of weight

The next step is to create a set of edges and weight, and arrange them in an ascending order of weightage (cost).

B, D	D, T	A, C	C, D	C, B	B, T	A, B	S, A	S, C
2	2	3	3	4	5	6	7	8

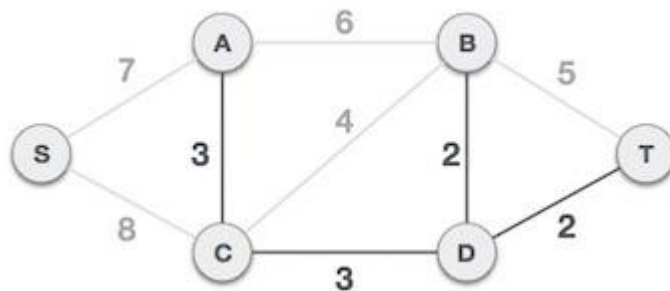
Step 3 - Add the edge which has the least weightage

Now we start adding edges to the graph beginning from the one which has the least weight. Throughout, we shall keep checking that the spanning properties remain intact. In case, by adding one edge, the spanning tree property does not hold then we shall consider not to include the edge in the graph.

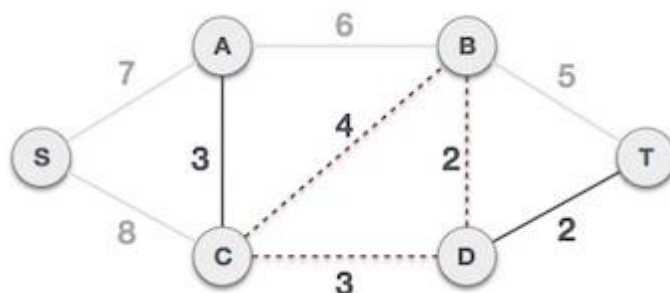


The least cost is 2 and edges involved are B,D and D,T. We add them. Adding them does not violate spanning tree properties, so we continue to our next edge selection.

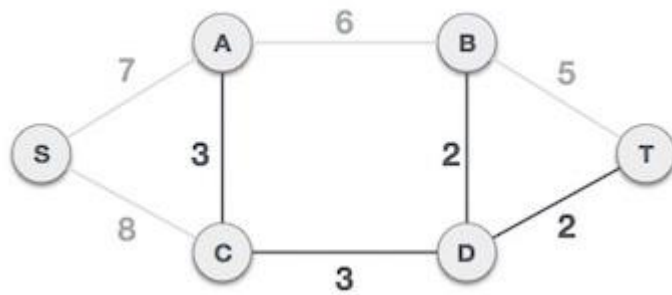
Next cost is 3, and associated edges are A,C and C,D. We add them again –



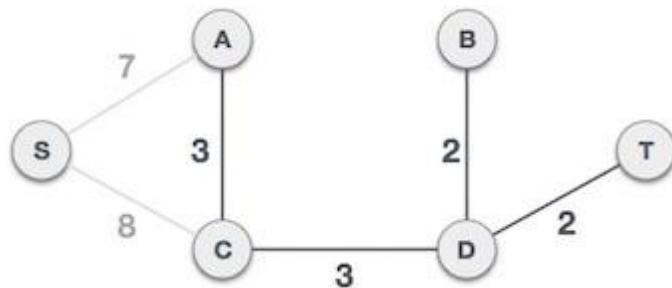
Next cost in the table is 4, and we observe that adding it will create a circuit in the graph. –



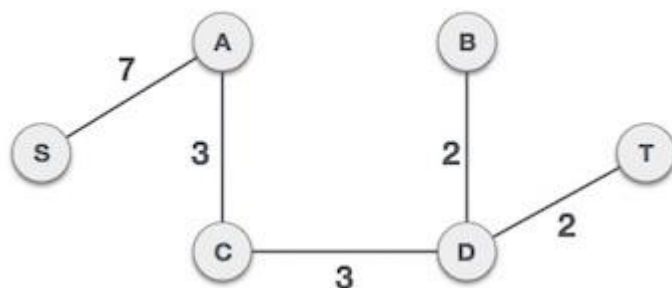
We ignore it. In the process we shall ignore/avoid all edges that create a circuit.



We observe that edges with cost 5 and 6 also create circuits. We ignore them and move on.



Now we are left with only one node to be added. Between the two least cost edges available 7 and 8, we shall add the edge with cost 7.



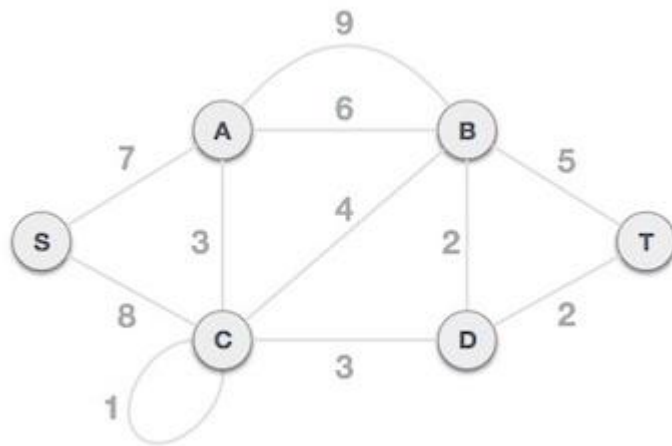
By adding edge S,A we have included all the nodes of the graph and we now have minimum cost spanning tree.

(1) Prim's Algorithm

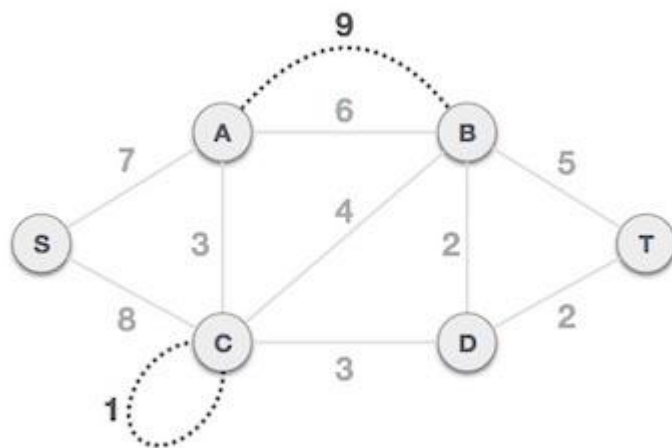
Prim's algorithm to find minimum cost spanning tree (as Kruskal's algorithm) uses the greedy approach. Prim's algorithm shares a similarity with the **shortest path first** algorithms.

Prim's algorithm, in contrast with Kruskal's algorithm, treats the nodes as a single tree and keeps on adding new nodes to the spanning tree from the given graph.

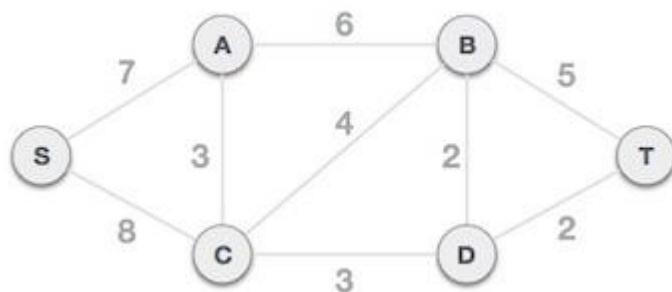
To contrast with Kruskal's algorithm and to understand Prim's algorithm better, we shall use the same example –



Step 1 - Remove all loops and parallel edges



Remove all loops and parallel edges from the given graph. In case of parallel edges, keep the one which has the least cost associated and remove all others.

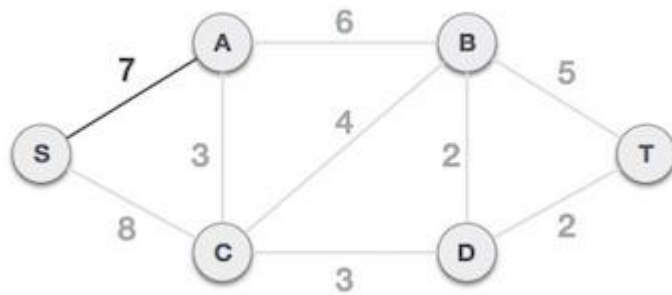


Step 2 - Choose any arbitrary node as root node

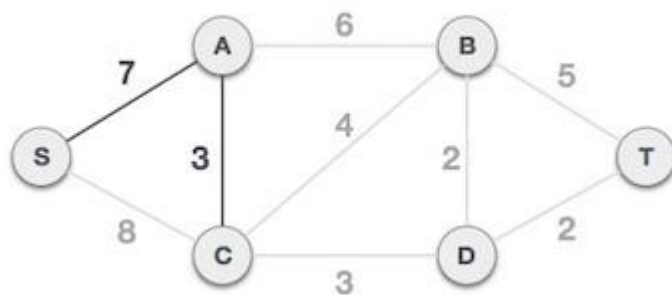
In this case, we choose **S** node as the root node of Prim's spanning tree. This node is arbitrarily chosen, so any node can be the root node. One may wonder why any node can be a root node. So the answer is, in the spanning tree all the nodes of a graph are included and because it is connected then there must be at least one edge, which will join it to the rest of the tree.

Step 3 - Check outgoing edges and select the one with less cost

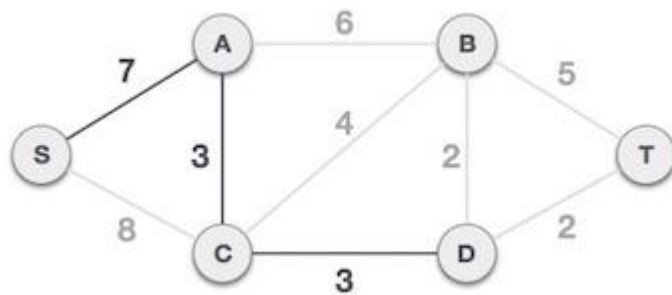
After choosing the root node **S**, we see that **S,A** and **S,C** are two edges with weight 7 and 8, respectively. We choose the edge **S,A** as it is lesser than the other.



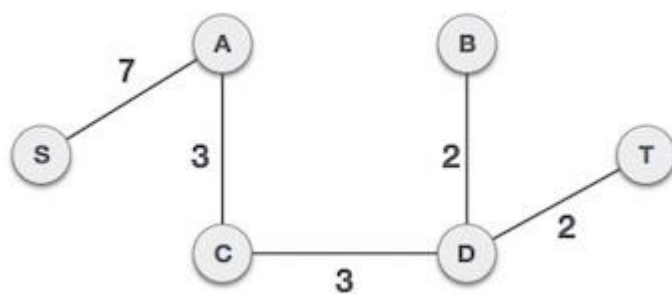
Now, the tree **S-7-A** is treated as one node and we check for all edges going out from it. We select the one which has the lowest cost and include it in the tree.



After this step, **S-7-A-3-C** tree is formed. Now we'll again treat it as a node and will check all the edges again. However, we will choose only the least cost edge. In this case, **C-3-D** is the new edge, which is less than other edges' cost 8, 6, 4, etc.



After adding node **D** to the spanning tree, we now have two edges going out of it having the same cost, i.e. **D-2-T** and **D-2-B**. Thus, we can add either one. But the next step will again yield edge 2 as the least cost. Hence, we are showing a spanning tree with both edges included.



We may find that the output spanning tree of the same graph using two different algorithms is same.

Dijkstra's Algorithm for Shortest Paths

Dijkstra's algorithm solves the single-source shortest-paths problem on a directed weighted graph $G = (V, E)$, where all the edges are non-negative (i.e., $w(u, v) \geq 0$ for each edge $(u, v) \in E$).

Example

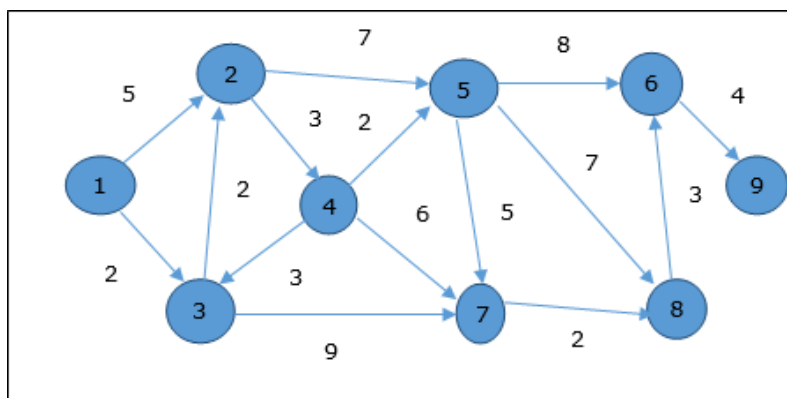
Let us consider vertex **1** and **9** as the start and destination vertex respectively. Initially, all the vertices except the start vertex are marked by ∞ and the start vertex is marked by **0**.

Vertex	Initial	Step1 V ₁	Step2 V ₃	Step3 V ₂	Step4 V ₄	Step5 V ₅	Step6 V ₇	Step7 V ₈	Step8 V ₆
1	0	0	0	0	0	0	0	0	0
2	∞	5	4	4	4	4	4	4	4
3	∞	2	2	2	2	2	2	2	2
4	∞	∞	∞	7	7	7	7	7	7
5	∞	∞	∞	11	9	9	9	9	9
6	∞	∞	∞	∞	∞	17	17	16	16
7	∞	∞	11	11	11	11	11	11	11
8	∞	∞	∞	∞	∞	16	13	13	13
9	∞	∞	∞	∞	∞	∞	∞	∞	20

Hence, the minimum distance of vertex **9** from vertex **1** is **20**. And the path is

$$1 \rightarrow 3 \rightarrow 7 \rightarrow 8 \rightarrow 6 \rightarrow 9$$

This path is determined based on predecessor information.

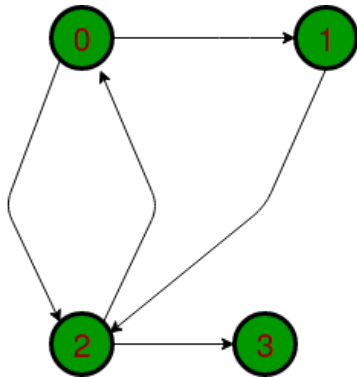


Transitive Closure of a Graph

Given a directed graph, find out if a vertex j is reachable from another vertex i for all vertex pairs (i, j) in the given graph. Here reachable mean that there is a path from vertex i to j . The reach-ability matrix is called transitive closure of a graph.

Example

Consider the below graph



Transitive closure of above graphs is

```

1 1 1 1
1 1 1 1
1 1 1 1
0 0 0 1

```

HASHING

Introduction to Hash Table

A **hash table** is a collection of items which are stored in such a way as to make it easy to find them later. Each position of the hash table, often called a **slot**, can hold an item and is named by an integer value starting at 0. For example, we will have a slot named 0, a slot named 1, a slot named 2, and so on. Initially, the hash table contains no items so every slot is empty. We can implement a hash table by using a list with each element initialized to the special Python value None. Figure 4 shows a hash table of size $m=11$. In other words, there are m slots in the table, named 0 through 10.

0	1	2	3	4	5	6	7	8	9	10
None	None	None	None	None	None	None	None	None	None	None

The mapping between an item and the slot where that item belongs in the hash table is called the **hash function**. The hash function will take any item in the collection and return an integer in the range of slot names, between 0 and $m-1$. Assume that we have the set of integer items 54, 26, 93, 17, 77, and 31. Our first hash function, sometimes referred to as the “remainder method,” simply takes an item and divides it by the table size, returning the remainder as its hash value ($h(\text{item}) = \text{item} \% 11$). Table 4 gives all of the

hash values for our example items. Note that this remainder method (modulo arithmetic) will typically be present in some form in all hash functions, since the result must be in the range of slot names.

Item	Hash Value
54	10
26	4
93	5
17	6
77	0
31	9

Once the hash values have been computed, we can insert each item into the hash table at the designated position as shown in Figure 5. Note that 6 of the 11 slots are now occupied. This is referred to as the **load factor**, and is commonly denoted by $\lambda = \text{number of items} / \text{table size}$. For this example, $\lambda = 6/11$.

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

Now when we want to search for an item, we simply use the hash function to compute the slot name for the item and then check the hash table to see if it is present. This searching operation is $O(1)$, since a constant amount of time is required to compute the hash value and then index the hash table at that location. If everything is where it should be, we have found a constant time search algorithm.

You can probably already see that this technique is going to work only if each item maps to a unique location in the hash table. For example, if the item 44 had been the next item in our collection, it would have a hash value of 0 ($44\%11=0$). Since 77 also had a hash value of 0, we would have a problem. According to the hash function, two or more items would need to be in the same slot. This is referred to as a **collision** (it may also be called a “clash”). Clearly, collisions create a problem for the hashing technique.

Static Hashing

In static hashing, the resultant data bucket address is always the same. In other words, the bucket address does not change. Thus, in this method, the number of data buckets in memory remains constant throughout.

Operations of static hashing are as follows.

Insertion – When entering a record using static hashing, the hash function (h) calculates the bucket address for search key (k), where the record will be stored. Bucket address = $h(K)$.

Search – When obtaining a record, the same hash function helps to obtain the address of the bucket where the data is stored.

Delete – After fetching the record, it is possible to delete the records for that address in memory.

Update – After searching the record using a hash function, it is possible to update that record. Furthermore, one major issue in static hashing is bucket overflowing. Some methods to overcome this issue are as follows.

Overflow chaining – New bucket created for the same hash result when the buckets are full

Linear Probing – Next free bucket allocated for data when a hash function generates an address where data is already stored.

Dynamic Hashing

An issue in static hashing is bucket overflow. Dynamic hashing helps to overcome this issue. It is also called **Extendable hashing method**. In this method, the data buckets increase and decrease depending on the number of records. It allows performing operations such as insertion, deletion etc. without affecting the performance.

Operations of dynamic hashing are as follows.

Insertion – Computes the address of the bucket. If the bucket is already full, it is possible to add more buckets. Moreover, it is possible to add additional bits to the hash value and re-compute the hash function. If the buckets are not full, it is possible to add data to the bucket.

Querying – Checks the depth value of the hash index and use those bits to compute the bucket address.

Update – Performs a query and update the data.

Delete – Performs a query to locate the desired data to delete.

The **main difference** between static and dynamic hashing is that, **in static hashing, the resultant data bucket address is always the same while, in dynamic hashing, the data buckets grow or shrink according to the increase and decrease of records.**

★★★★★★

UNIT-V

FILES AND ADVANCED SORTING

FILE ORGANIZATION

A file is a sequence of records. File organization refers to physical layout or a structure of record occurrences in a file. File organization determines the way records are stored and accessed. In many cases, all records in a file are of the same record type. If every record in the file has exactly the same size (in bytes), the file is said to be made of fixed-length records. If different records in the file have different sizes, the file is said to be made up of variable-length records.

Here it is worthwhile to note the difference between the terms file organization and access method. A file organization refers to the organization of the data of a file into records, blocks and access structures; this includes the way the records and blocks are placed on the storage medium and interlinked. An access method on the other hand, provides a group of operations – such as find, read, modify, delete etc., — that can be applied to a file. In general, it is possible to apply several access methods to a file organization. Some access methods, though, can be applied only to files organised in certain ways. For example, we cannot apply an indexed access method to a file without an index.

Types of File Organization

There are three types of organizing a file

1. Sequential access file organization
2. Direct access file organization
3. Indexed sequential access file organization

1. Sequential access file organization

- Storing and sorting in contiguous block within files on tape or disk is called as **sequential access file organization**.
- In sequential access file organization, all records are stored in a sequential order. The records are arranged in the ascending or descending order of a key field.
- Sequential file search starts from the beginning of the file and the records can be added at the end of the file.
- In sequential file, it is not possible to add a record in the middle of the file without rewriting the file.

Advantages

- It is simple to program and easy to design.
- Sequential file is best use if storage space.

Disadvantages

- Sequential file is time consuming process.
- It has high data redundancy.
- Random searching is not possible.

2. Direct access file organization

- Direct access file is also known as random access or relative file organization.
- In direct access file, all records are stored in direct access storage device (DASD), such as hard disk. The records are randomly placed throughout the file.
- The records does not need to be in sequence because they are updated directly and rewritten back in the same location.
- This file organization is useful for immediate access to large amount of information. It is used in accessing large databases.
- It is also called as hashing.

Advantages

- Direct access file helps in online transaction processing system (OLTP) like online railway reservation system.
- In direct access file, sorting of the records are not required.
- It accesses the desired records immediately.
- It updates several files quickly.
- It has better control over record allocation.

Disadvantages

- Direct access file does not provide back up facility.
- It is expensive.
- It has less storage space as compared to sequential file.

3. Indexed sequential access file organization

- Indexed sequential access file combines both sequential file and direct access file organization.
- In indexed sequential access file, records are stored randomly on a direct access device such as magnetic disk by a primary key.
- This file have multiple keys. These keys can be alphanumeric in which the records are ordered is called primary key.
- The data can be access either sequentially or randomly using the index. The index is stored in a file and read into memory when the file is opened.

Advantages

- In indexed sequential access file, sequential file and random file access is possible.
- It accesses the records very fast if the index table is properly organized.
- The records can be inserted in the middle of the file.
- It provides quick access for sequential and direct processing.
- It reduces the degree of the sequential search.

Disadvantages

- Indexed sequential access file requires unique keys and periodic reorganization.
- Indexed sequential access file takes longer time to search the index for the data access or retrieval.
- It requires more storage space.
- It is expensive because it requires special software.
- It is less efficient in the use of storage space as compared to other file organizations.

ADVANCED SORTING**Sorting on Several keys**

Sorting is a process of ordering or placing a list of elements from a collection in some kind of order. It is nothing but storage of data in sorted order. Sorting can be done in ascending and descending order. It arranges the data in a sequence which makes searching easier.

For example, suppose we have a record of employee. It has following data:

Employee No.
Employee Name
Employee Salary
Department Name

Here, employee no. can be takes as key for sorting the records in ascending or descending order. Now, we have to search a Employee with employee no. 116, so we don't require to search the complete record, simply we can search between the Employees with employee no. 100 to 120.

Similarly, More advanced sorting can be done by making use of more than one key such as both Employee No. & Employee Name(two keys).

Summary of Internal Sorting & External Sorting

The techniques of sorting can be divided into two categories. These are:

- ✓ Internal Sorting
- ✓ External Sorting

Internal Sorting:

If all the data that is to be sorted can be adjusted at a time in the main memory, the internal sorting method is being performed.

An **internal sort** is any data sorting process that takes place entirely within the main memory of a computer. This is possible whenever the data to be sorted is small enough to all be held in the main memory. For sorting larger datasets, it may be necessary to hold only a chunk of data in memory at a time, since it won't all fit. The rest of the data is normally held on some larger, but slower medium, like a hard-disk.

Any reading or writing of data to and from this slower media can slow the sortation process considerably. This issue has implications for different sort algorithms.

Some common internal sorting algorithms include:

1. Bubble Sort
2. Insertion Sort
3. Quick Sort
4. Heap Sort
5. Radix Sort
6. Selection sort

External Sorting:

When the data that is to be sorted cannot be accommodated in the memory at the same time and some has to be kept in auxiliary memory such as hard disk, floppy disk, magnetic tapes etc, then external sorting methods are performed.

External sorting is a term for a class of sorting algorithms that can handle massive amounts of data. External sorting is required when the data being sorted do not fit into the main memory of a computing device (usually RAM) and instead they must reside in the slower external memory (usually a hard drive). External sorting typically uses a hybrid sort-merge strategy. In the sorting phase, chunks of data small enough to fit in main memory are read, sorted, and written out to a temporary file. In the merge phase, the sorted sub-files are combined into a single larger file.

One example of external sorting is the external merge sort algorithm, which sorts chunks that each fit in RAM, then merges the sorted chunks together. We first divide the file into **runs** such that the size of a run is small enough to fit into main memory. Then sort each run in main memory using merge sort sorting algorithm. Finally merge the resulting runs together into successively bigger runs, until the file is sorted.

★★★★★★